

Optimizing Federated Learning With Aggregation Strategies: A Comprehensive Survey

NAEEM KHAN ¹, SHIBLI NISAR ¹, MUHAMMAD ASGHAR KHAN ² (Senior Member, IEEE),
YASAR ABBAS UR REHMAN³, FAZAL NOOR ⁴, AND GORDANA BARB ⁵ (Associate Member, IEEE)

¹Department of Electrical Engineering, Military College of Signals, National University of Sciences and Technology, Rawalpindi 46000, Pakistan

²Department of Electrical Engineering, Prince Mohammad Bin Fahd University, Al Khobar 31952, Saudi Arabia

³TCL Corporate Research (HK) Company, Ltd., New Territories, Hong Kong

⁴Faculty of Computer and Information Systems (FCIS), Islamic University of Madinah, Madinah 42351, Saudi Arabia

⁵Department of Communications, Politehnica University Timisoara, 300006 Timisoara, Romania

CORRESPONDING AUTHOR: GORDANA BARB (e-mail: gordana.barb@upt.ro).

ABSTRACT This article provides a comprehensive survey of aggregation strategies in federated learning (FL). This decentralized machine learning (ML) paradigm enables multiple clients to collaboratively train models without sharing their local datasets. Aggregation is a pivotal aspect of FL, as it integrates model updates from diverse clients into a unified global model while addressing critical challenges such as data heterogeneity, scalability, and privacy preservation. The study categorizes aggregation strategies into three primary approaches: data-centric, model-centric, and secure aggregation, each tailored to address distinct problems in FL systems. Data-centric strategies focus on addressing non-independent and identically distributed (non-IID) data across clients, ensuring that model updates account for imbalances in data distributions. Model-centric strategies optimize the aggregation of model parameters, emphasizing techniques such as weighted averaging and model distillation to improve model performance across clients with diverse data characteristics. Secure aggregation techniques aim to enhance privacy and robustness, protecting client data from potential adversarial threats through encryption-based methods like secure multi-party computation (SMPC) and Byzantine-resilient techniques. The analysis delves into the advantages and limitations of these aggregation strategies, particularly their role in tackling challenges like non-IID data, communication efficiency, and resistance to adversarial attacks. Furthermore, the article identifies existing research gaps in FL, including the need for more scalable and robust aggregation methods capable of reducing communication costs, enhancing privacy guarantees, and improving performance in highly heterogeneous environments. These insights provide a roadmap for future research aimed at advancing aggregation strategies in FL to improve model accuracy, security, and efficiency in real-world applications.

INDEX TERMS Federated learning, aggregation strategies, weighted averaging, model distillation, data heterogeneity, Byzantine-resilient aggregation, privacy preservation, communication efficiency, non-IID data, secure aggregation.

I. INTRODUCTION

The FL can be regarded as a novel foundational technology under artificial intelligence. The proposal has been first introduced to the public by Google in 2017 to target the problems arising from the local model training and updates in human mobile edge devices [1], [2], [3]. The aim of FL is to achieve worthwhile collaborative ML among several individuals or

computing units and protect information integrity during data sharing in large-scale multi-tasking. This includes, inter alia, protection of terminal and personal data and legal compliance. FL outlines immense prospects as the basic model of the future AI collective algorithms and the network [4].

FL provides ML architecture to overcome the original problem of collaboration among multiple data owners while

sharing no data [1]. The global model means the best setting that includes the knowledge incorporated by all the participants. FL requires that the modeling outcome must closely mimic the normal pattern, implying that the data of multiple owners need to be aggregated by one owner for the purpose of modelling [5]. Since data is not being transferred, user privacy is not violated, and the essential data specifications that meet various legal reporting standards are not influenced either, including general data protection regulation (GDPR). In this article, the system architecture of FL is illustrated which includes two data owners, which for simplicity are referred to as edge A, and edge B. The system is also extensible to the case where there are multiple edge data owners, and it can also be extended to accommodate additional use cases. Suppose edge A and edge B both decide to perform joint training of an ML model to employ the relevant user data they can access within their business applications. In scenarios where A and B are permitted to exchange data directly, particularly due to concerns regarding data privacy and security, the implementation of an FL system can be utilized to construct the model. The current landscape of deployments employs a single-server centralized architecture. This approach is susceptible to risks associated with component failure, system scalability, and communication efficiency, among others [4]. The identified issues will hinder the continuous operability of the AI/ML components in real-world industrial applications, potentially compromising the system and resulting in severe accidents.

A. IMPORTANCE OF AGGREGATION IN FL

FL is a decentralized ML approach where multiple clients collaborate to train a global model without sharing their local data. Four key issues have been identified to support FL, and the first is aggregation since it involves the coordination of clients to combine their updates to create a global model. Its importance lies in several key aspects:

1) PRIVACY PRESERVATION

The process of aggregation means that local data can be used without sending it to the central server, making it possible to fulfill privacy legislation [2]. Parameters remain the only information being transmitted; gradient information prevents sensitive information from being transmitted.

2) SCALABILITY

FL's scalability can be extended to several devices or clients within a system, each contributing to a model [2]. Strong aggregation methods allow the global model to incorporate updates from thousands or even millions of clients without overloading the system.

3) MODEL ACCURACY

The updates collected from a variety of sources help in global model learning instead of separate localized models [2]. This leads to enhanced accuracy, particularly when working with non-IID.

4) HANDLING HETEROGENEITY

Many clients in FL can have different data and computational power. Aggregation algorithms are prominent when it comes to guaranteeing that the contributions of different clients with different distributions of the provided data or differing computational capabilities will be considered with comparable weights, thus admitting fair learning in the overall distributed system [3].

5) FAULT TOLERANCE

Aggregation methods provide robustness to the FL system by mitigating the effects of unreliable clients (e.g., devices dropping out) and noisy updates [2], [3]. By averaging or adjusting the weights of updates, aggregation techniques maintain model stability.

6) COMMUNICATION EFFICIENCY

Since FL requires periodic communication between clients and the server, aggregation techniques can reduce communication costs by effectively compressing the model updates or incorporating client updates asynchronously [4].

Thus, the aggregation process is central to FL's functioning and success, enabling privacy-preserving, scalable, and accurate distributed learning.

B. OVERVIEW OF AGGREGATION STRATEGIES

Aggregation strategies in FL can be categorized into three main types: data-centric, model-centric, and robust and secure aggregation. These strategies solve different issues that emerge because of FL's decentralized structure, the diversity of the client datasets, and the necessity of efficient and secure information exchange.

Data-centric aggregation is concerned with how to process client updates in an interoperable way depending on the nature of data at the local level. In simple averaging, the most unsophisticated approach, each client's contribution to the final global model is proportional to the rate of sending updates, suggesting that all clients have the same amount of data and data is IID [5], [6], [7], [8], [9], [10]. However, in many scenarios for FL, the data is non-IID, and the clients have different amounts of data. To this end, weighted averaging is employed, and one of its more popular implementations is known as Federated Averaging (FedAvg) which is explained in detail in the next sub-section. In FedAvg the global model is updated through aggregating local models where the contributions for averaging are weighted by the client dataset size thus favoring clients with large datasets. This also helps with the final steps of averaging the overall model with a more diverse and aligned dataset [11], [12].

Adaptive averaging goes an extra step further from the process of aggregation in that it considers the quality or relevance of the local data. FedAvg with personalization (FedAvg-Pers) provides a degree of customization from the global model at the client level to allow clients to keep a form of localized model. This is especially useful where client data distributions

are quite different as they prevent the clients' performance from being affected by these few data points. Another adaptive technique includes Data-Aware Aggregation (Dawa), which incorporates quality of data into the factors used in aggregation of the client's updates so that the updates of clients with better quality of data are given more weightage when computing an aggregated global model. Local data-aware aggregation (L-Dawa) builds on this idea but enforces even more local adjustments to cope with the non-IID nature of the data on clients [3]. These strategies enhance the viability and versatility of the coarse-grained global model in various data settings by being more sparing in how client updates are blended.

But model-centric aggregation, in contrast, lies in understanding how such model updates for the next iteration themselves can be accumulated or modified. Model distillation, for instance, fuses knowledge from several local models rather than averaging their weight. It can do this particularly well in the case of models trained on heterogeneous data because it transfers the core knowledge from each client model into an aggregate global model. Federated distillation (FD) is a particular case of model distillation for FL where, instead of sending the full model, clients share distilled information, e.g., class outputs or logits [13], [14], [15]. This lessens the information exchange load without incurring losses of efficacy. The second model-centric approach is model compression, where the size of updates is reduced prior to their actual transmission. Making the gradients thinner and quantizing the gradients are some of the methods used to compress the model updates to reduce the communication cost, but without significantly impacting the global model's performance.

Besides, other methods used in FL are knowledge transfer methods, which are primarily concerned with how to transfer the knowledge derived from the local learning process to the global model; FL often applies teacher-student frameworks for this purpose. This helps clients carry information to share without passing the full models or some valuable information that might time be valuable [16], [17]. Other central techniques for model-centric methods focus on efficiency; methods such as gradient sparsification, asynchronous updates, and multi-task learning to enable high efficiency aggregation in large-scale federations. To achieve these, advanced message-passing techniques that optimize both communication and the model across clients are used to ensure FL systems can accommodate millions of clients [18], [19], [20], [21], [22], [23].

C. RESEARCH GAPS AND MOTIVATIONS

FL has recently emerged as an effective privacy-preserving approach to decentralized learning, nevertheless, several gaps of knowledge that hamper the growth and application of FL have remained unseen. One important topic, which needs more research, is data heterogeneity management. Non-IID data across clients is one of the simplest, yet complex issues in

FL since it has large implications on model convergence, accuracy, and fairness. Specifically, existing aggregation strategies including weighted and adaptive averaging partially solve this problem; however, there remains a gap to learn more sophisticated methods for handling highly diverse data contributions from clients without significantly degrading the overall model performance. This leads to the analysis of improved and special aggregation methods compared to the standard averaging that gives attention to clients with diverse distributions of data while providing the best performance on the global model.

A key research gap in FL involves reducing communication overhead, which limits scalability in real-world settings despite partial solutions like model compression. While secure methods such as SMPC and homomorphic encryption (HE) offer privacy, they introduce significant computational costs, necessitating lightweight yet robust aggregation alternatives. Future work should explore dynamic hybrid aggregation approaches that adapt to varying security threats and data characteristics, enabling flexible and efficient deployment in constrained environments.

Another missed approach is a lack of unification and transferability of procedures to formulate aggregation strategies in general for various sorts of models and learning problems. Most of the current techniques are designed in an application-specific setting or for a peculiar structural configuration of the target models such as the Convolutional Neural Network (CNN) or for straightforward classification problems. However, as FL is being applied to more tasks such as natural language processing, time-series forecasting, and reinforcement learning, it is incumbent that there are aggregation methodologies that can be applied across the numerous forms of FL and domains. The motivation here is to develop better and more generalizable strategies of aggregation so that FL can be deployed in more domains where it is needed, including health care, finance, etc.

Recent studies such as [128] and [129] demonstrate the importance of balancing privacy, efficiency, and model performance in FL aggregation. However, there exists a lack of a structured decision-making framework that helps practitioners select optimal aggregation methods based on deployment-specific constraints such as computational resources, network heterogeneity, or security requirements. This study aims to fill that gap by consolidating aggregation strategies and presenting a comparative analysis grounded in recent state-of-the-art contributions.

Finally, many current aggregation methods are predicated on a central server scenario which can represent a single point of failing and thus weakness. This makes there an increasing interest in adopting decentralized or hierarchical aggregation strategies that disaggregate the coverage process across multiple servers or edge devices. They could improve the reliability and modularity of FL systems to encourage fault tolerance without jeopardizing the need for a central authority. This may also enhance system latency, which can reduce the

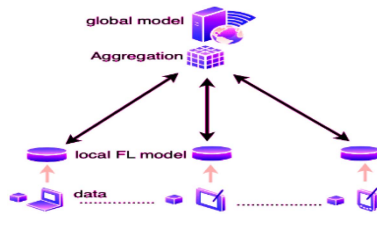


FIGURE 1. Basic FL architecture.

applicability of FL to real-time applications, with the decentralized aggregation.

II. BACKGROUND

This section provides an overview of the fundamentals of FL, including its architecture, key challenges, and how aggregation plays a role in addressing these challenges.

A. FL ARCHITECTURE

Over the last few years, a lot of work has been done in FL algorithms and models, primarily thanks to ubiquitous training data harvested from moveable gadgets including smart mobile phones and self-driving cars. AI has developed because lots of data is readily available and has pressed the pace of technological innovation. Among them, privacy issues have become the most important problem. Consumers are now increasingly shy to share personal information, especially data such as records and shopping patterns known as health-privacy concerns resulting from increased awareness of the risks of proliferation of privacy violations [6], [7], [8].

Currently, as the data is accumulated the storage is gradually integrated into data silos, which means that data originating from different devices is stored separately to ensure privacy and security of client data [6]. Although this approach safeguards the user's privacy, it has immense problems when it comes to data accessibility. As these challenges demonstrate, data silos make the collection of sufficient data for training deep learning models increasingly difficult. Therefore, within the context of data silos, the traditional deep learning framework is insufficient.

Fig. 1 shows the basic architecture of FL. The typical FL architecture consists of three key components: clients, or more specifically, end devices, a central server, and a communication channel. Clients, which may be mobile phones, an IoT device, or an edge node, perform the model update calculation based on the data that is kept locally. Aggregation of the updates obtained from the clients as well as the updating of the global model is handled by the central server [7]. Through the communication network, there is the transfer of model updates from the client side to the central server, which may employ secure channels to uphold the privacy of the information as well as its correctness.

In an FL system, typically a client initiates its action by first receiving a copy of the global model from the central server. The clients then use the locally generated values of the data to perform an update to the model, using the computed gradients.

After the local training is done the clients upload their model updates (frequently in the form of gradients or model weights) to the central server [6], [8]. The update information will be combined via several predefined aggregation schemes like simple average, weighted average, or more sophisticated ones, and afterwards the combined update will be applied at the global model located at the central node. These steps are taken over several communication cycles to continue refining the global model until an acceptable performance level is attained.

Advanced FL architectures increasingly incorporate mechanisms like HE, Secure Multiparty Computation (SMPC), and personalized learning layers to adapt to user-specific data distributions while preserving privacy [128], [129]. Furthermore, in response to user and regulatory demands, evolving architectures consider zero-shot and transfer learning-based modules that decouple model generalization from localized data availability.

One of the defining features of FL architecture is therefore its capacity to accommodate the inherent heterogeneity with regards to the data sets employed and the learning devices utilized. In a federated system, clients typically possess non-IID data, meaning their local datasets differ in distribution and may also vary in size [7], [8]. Further, clients may have different computer powers, meaning that they are able to perform a certain number of local updates before having to send them to the central server. FL architecture also must be sufficiently flexible to handle such variations and, at the same time, guarantee high-fidelity model training.

III. TYPES OF AGGREGATION STRATEGIES

In this section, we discuss the various types of aggregation strategies, and within each type, sub-types are also presented to enhance understanding.

A. DATA-CENTRIC AGGREGATION

1) SIMPLE AVERAGING

Simple averaging is one of the base-level methodologies of FL since the global model is updated by averaging the stochastic models derived by each client. This method is a variant of FL, which assumes that all the clients send their updated model to the central server – either in part or in full – without considering the quantity or quality of data that each client possesses. In each iteration where local training is finished, each of the clients contributing her parameters (weight, for instance) receives an average model, which goes back to the next round of training.

The principle of operation of simple averaging suggests that all the clients provide the same level of contribution. Because all the client updates are equal, the basic method of averaging is quite effective in cases based on the assumption of IID data of clients [25]. Although in IID scenarios where all the clients are distributed similarly this approach can deteriorate performance because the global model might not capture all the patterns well enough. Simple averaging remains popular due to its simplicity and work efficiency regardless of the

unreliability when used for applications in a heterogeneous structure. Thus, simple averaging remains popular due to its simplicity and work efficiency irrespective of the aforesaid shortcomings when used for applications in an environment with small discrepancies in client data and computational resources.

Let ω_i represent the local model weights from client i . The global model update ω_t at round t is given by:

$$\omega_t = \frac{1}{N} \sum_{i=1}^N \omega_i \quad (1)$$

Where:

- N is the total number of participating clients,
- ω_i is the model weight from client i ,
- ω_t is the updated global model at time t .

2) WEIGHTED AVERAGING

Another way of aggregation in FL is a more advanced form of weighted averaging that takes into consideration that the amount and quality of data gathered by each client might differ significantly. While simple averaging means integrating the contribution of all the clients in developing the global model, weighted averaging assigns weight to the client according to certain factors such as size of the local dataset. This helps to guarantee clients with potentially larger datasets have a higher impact on updating the global model onto one that holds a distribution that is closer to that of all clients' data.

The key principle of weighted averaging lies in its proportional contribution. By assigning weights that reflect the number of local data points or other factors like model accuracy, clients with more significant data (or better-quality data) have a proportionately larger impact on the global model. This approach is beneficial in non-IID scenarios, where clients' data distributions are not uniform. Weighted averaging helps mitigate the bias introduced by clients with small or unrepresentative datasets, ensuring that the global model is more robust and better captures the diversity of data across the network.

Let n_i be the number of local data samples on client i , and N be the total number of samples across all clients. The global model update ω_t at round t is calculated as:

$$\omega_t = \frac{\sum_{i=1}^N n_i \omega_i}{\sum_{i=1}^N n_i} \quad (2)$$

Where:

- n_i is the number of data samples from client i ,
- ω_i is the local model weight from client i ,
- ω_t is the updated global model at time t ,
- The numerator ensures that clients with more data make a larger contribution to the global model.

B. FEDAVG

Another widely used algorithm that naturally applies weighted averaging in FL is federated averaging, or FedAvg as shown in Fig. 2. The contribution of each client to the global



FIGURE 2. FedAvg architecture.

model in FedAvg is proportional to the number of data points possessed by a client. This method has it that clients dealing with large data sets contribute more to the global model which is useful especially where clients' data vary greatly. This technique maintains the proportionate communication complexity and model refinement making FedAvg one of the most popular aggregation techniques in FL [25]. This minimizes communication costs since the client trains for several updates locally before sending only the result of all updates, which is averaged at the server. This is helpful in extending FL to sparsely populated areas by many clients and where the amount of data transmitted is different.

The FedAvg process begins with a global model which has been initialized and a copy of which is maintained on each device. Next, each device performs model updating locally by using its own data, while the global model, which is retained by each device, is initially used for reference purposes. After local training, the devices send the new models to the central server for compilation to a consolidated model. The new models are accumulated at the central server that computes their average using weights assigned to them. The mechanism ensures that the devices with a greater amount of data assigned have a greater influence on the final model. The model that is generated globally is then sent back to the devices and the process repeats until it has converged [26].

The mathematical formula for FedAvg can be described as follows:

Algorithm: FedAvg Algorithm.

```

1 Algorithm FederatedAveraging (K, R)
2 while K ≠ length (ConnectedClients()) do
3   Loop
4   end while
5   FedAvg ()
6   ReleaseClients ()
7   Procedures FedAvg ()
8   Initialize global model parameters:  $\theta_0$ 
9   for t = 1, R do
10      $S_t \leftarrow \text{Subset}(K, \text{"random"})$ 
11     for k ∈  $S_t$  do
12       Receive global model parameters:  $\theta_t$ 
```

```

13 Train on client  $k$ 's dataset:  $\theta^k$ 
14 Compute local update:  $\delta\theta_t^k = \delta\theta_t^k - \theta_t$ 
15 Send local update to server:  $\delta\theta_t^k$ 
16 end for
17 Receive local updates:  $\delta\theta_t^k$ 
18 Compute aggregated update:  $\Delta\theta_t = \frac{(\sum_k \delta\theta_t^k)}{K}$ 
19 Update global model parameters:  $\theta_{t+1} = \theta_t + \Delta\theta_t$ 
20 end for

```

Another major feature of FedAvg is that it decreases communication complexity because clients have local computations on their own and send updates only occasionally. This approach therefore minimizes the number of communications needed, especially in many client settings [29]. The process repeats local training, aggregation, and redistribution until local model desires, or the global model is deemed satisfactory or converged.

Due to asynchronous training, some clients might train on an outdated version of the global model, which can lead to model staleness and degrade overall performance. This is particularly problematic in environments with high communication delays or unreliable client connections.

1) ADAPTIVE AVERAGING

Adaptive Averaging is a complex aggregation approach implemented in FL that is the same as the above where the weights that are attributed to all local models during the aggregation process are based on certain conditions. Unlike simple or weighted averaging, where the weights always are very fixed, like the size of the local dataset, adaptive averaging tunes the weights by considering other attributes like data distribution, model accuracy, and the relative contribution to the global update each client provides [24], [25]. This method is most useful in FL setup since the client's devices differ in terms of data quality and processing power.

By continuously adapting the aggregation process, the global model is refined in a way that accounts for discrepancies in data distribution, client updates, and resource availability.

The size of the clients' datasets is not necessarily equal to central node as $\omega_{t+1} \leftarrow \omega_t - \eta \sum_{k \in K_t} \frac{n_k}{n} g_k^t$, where η is the learning rate and $g_k^t = \Delta\omega_t L_k(\omega_t)$, and $n_0 = P_k \subset K_t \cdot n_k$. However, this approach is not efficient in communication between the central node and the clients. More efficient implementations can be achieved using FedAvg [1], where the aggregation in the central node is computed as $\omega_{t+1} \leftarrow \sum_{k \in K_t} \frac{n_k}{n} \omega_{t+1}^k$. In this case, the model updates from each client ω_{t+1}^k can be computed locally over several iterations of gradient descent, with $\omega_k \leftarrow \omega_k - \eta \Delta\omega_k L_k(\omega_k)$.

In distributed optimization the shards of the clients are assumed to be IID, whereas federated optimization does not necessarily rely on this assumption. FL algorithms can be trained with gradient descent strategies by updating the parameters of the model from the local updates provided by

the clients. Thus, at each iteration t , a subset of $K_t \subset K$ clients is selected for synchronous aggregation, i.e., the central node responsible for computing the aggregated model sends the latest update of the model to the selected clients. Then, these clients compute local updates of the model using their own shards (e.g., by computing the gradients of the loss function with respect to the model parameters) and send the corresponding local updates to the central node, which updates the model according to a defined aggregation rule [33]. Typical implementations use gradient descent and compute the model updates in the central node as central node as $\omega_{t+1} \leftarrow \omega_t - \eta \sum_{k \in K_t} \frac{n_k}{n} g_k^t$, where η is the learning rate and $g_k^t = \nabla\omega_t L_k(\omega_t)$, and $n_0 = p_{k \in K_t} n_k$.

FedAvg updates the model through the calculation of a weighted average of the updates contributed by various clients, factoring in the quantity of training points supplied by each client. However, all these techniques lack fault tolerance and may exhibit fragility when exposed to malicious or faulty clients. It has been demonstrated that the presence of a single problematic client can compromise the entire federated optimization algorithm, resulting in suboptimal solutions or convergence issues [34]. To address these limitations, Adaptive FedAvg (AFA) is introduced, an algorithm designed for FL that is resilient to faulty, noisy, and malicious clients. Unlike FedAvg, our algorithm enhances the model by considering both the volume of data supplied by each client and their reliability, which is assessed through the estimation of the probability that each client will deliver beneficial model updates. Thus, at each iteration, given a set of selected clients K_t that compute their model updates ω_{t+1}^k , the central node updates the model as:

$$\omega_{t+1} \leftarrow \sum_{k \in K_t^g} \frac{p_{k_t} n_k}{N} \omega_{t+1}^k \quad (3)$$

where p_{k_t} is the probability of client k providing good model updates (according to AFA's aggregation rule) at iteration t and $N = \sum_{k \in K_t^g} p_{k_t} \eta n_k$. The subset $K_t^g \subset K_t$ contains the clients that provide a good update according to the robust aggregation criteria. The adaptive federated optimization framework incorporates server-side momentum and adaptive learning rate mechanisms to enhance model performance across heterogeneous clients [121].

Adaptive averaging offers key advantages in FL by addressing non-IID data and client heterogeneity through dynamic weighting of local updates, enhancing global model robustness against skewed or imbalanced datasets [35]. It also accommodates computational variability, allowing clients with higher resources to contribute more significantly, thus mitigating bottlenecks caused by slower devices [36], [37]. Unlike static averaging, adaptive methods adjust aggregation parameters during training, improving convergence and ensuring the global model reflects diverse data distributions [38]. For instance, in healthcare or edge computing scenarios where data varies widely across nodes, adaptive averaging balances contributions proportionally, reducing bias from outliers or

dominant clients [36], [39]. This leads to more accurate and reliable FL outcomes in heterogeneous environments.

2) FEDAVG WITH PERSONALIZATION (FEDAVG-PERS)

FedAvg-Pers is an enhancement of the basic FedAvg algorithm specially developed to overcome the shortcomings of traditional FL in real-world decentralized settings especially when the data distributed across clients is not non-IID. FedAvg-Pers extends the FedAvg framework by introducing client-level personalization alongside global model training. While FedAvg assumes a single global model suffices for all clients, FedAvg-Pers allows each client to maintain a personalized model tailored to its local data distribution, improving local relevance and generalization. Clients update both global and private parameters during training, synchronizing only the global components with the server. The server then aggregates these global components and broadcasts the updated model, preserving local adaptations. This design balances global consistency with individual specificity, enhancing convergence and performance under non-IID data settings. Compared to more complex methods like MOCHA, FedAvg-Pers offers a lightweight alternative with practical implementation advantages. It is particularly effective in applications with high data heterogeneity, such as personalized healthcare, recommender systems, and edge-based smart devices. Recent personalized extensions of FedAvg aim to address challenges associated with data heterogeneity [128]. For example, pFedMe introduces a bilevel optimization framework using Moreau envelopes to preserve personalization while maintaining generalization. MOON leverages contrastive learning to enforce alignment between global and local models. Meanwhile, FedBABU separates feature extraction and classification stages, fine-tuning only the classifier to adapt to local domains without disrupting shared representations. These approaches demonstrate improved performance over standard FedAvg in non-IID settings.

3) DAWA (DATA-AWARE AGGREGATION)

Data-Aware Aggregation or DAWA is a novel aggregation approach to deal with the issues inherent in FL including the drift of clients' models due to data heterogeneity. It presents the notion of angular difference between what the local model for a client and the global model [40]. This dispersion is used as a factor for scaling the share of each client's model during the aggregation. DAWA achieved the goal proposed in earlier works that is, prevent client control of the optimization trajectory to minimize the impact of large client dataset size or when the client's data distribution is different from other clients.

Data-Aware Aggregation focuses on adjusting the contribution of each client to the global model based on the quality and distribution of its local data. Nishio & Yonetani proposed a data-aware client selection mechanism that prioritizes clients based on their computational capabilities and data distributions, ensuring better aggregation efficiency [122]. Utilizing

this concept, Wang et al. introduced DAWA, where clients with more representative data contribute more to the global model, improving robustness against non-IID data [124].

The primary concept of DAWA is to decrease the impact of the divergence of the client model during the aggregation process of FL. Due to such reasons clients in FL may have highly non-IID data and hence experience a substantial level of model drift from the global model. DAWA computes the cosine similarity between each client's model and an amalgamate model wherein the angular divergence of the two weight vectors is obtained [41]. This divergence is then utilized in aggregation to lessen the influence of clients which have models far from the organization's global direction on the global model's update. This way, angular divergence acts as a signal that guides DAWA in keeping the optimization trajectory free from disturbance by certain categories of clients while still heading toward the state of convergence of the global model. The angular divergence is given as [42] :

$$\delta_k = \cos\theta_{g,k} = \frac{\omega_g^r \cdot \omega_k^r}{\|\omega_g^r\| \|\omega_k^r\|} \quad (4)$$

Integration of angular divergence requires $\beta_k = \delta_k/K$ thus yielding equation:

$$\omega_g^{r+1(M-DAWA)} = \frac{1}{K} \sum_{k=1}^K \sigma_k \omega_k^r \quad (5)$$

One of the main benefits of DAWA is found in cases where data heterogeneity is a problem in the context of employed clients. DAWA solves the problem of getting bias when some clients have more data or data are highly divergent than other clients, by offering a divergence aware weighting mechanism. This results in more stable model training as well as improved ability to achieve high results on downstream tasks. DAWA is especially beneficial in cross-silo learning since all client clients contribute to every round of training and cross-device setup where a subset of clients engages in training [40], [41]. The method accelerates convergence and improves the overall accuracy of the global model, especially when combined with self-supervised learning approaches in federated settings, such as SimCLR and Barlow Twins, as demonstrated in studies with CIFAR-10/100 and Tiny ImageNet datasets.

4) L-DAWA (LAYERWISE DATA-AWARE AGGREGATION)

L-DAWA is an extension of DAWA: in addition to each layer's covariance matrix, L-DAWA performs a layer-wise angular divergence calculation. While DAWA gives a single value of divergence of the entire model, L-DAWA calculates the angular divergence of each layer in isolation. It permits more precise control over the aggregating since various strata of a model might diverge sharply in clients. Layer-wise divergence can be calculated using [42]:

$$\omega_g^{r+1(L-DAWA)} = \frac{1}{K} \sum_{l=1}^L \sum_{k=1}^K \delta_k^{(l)} \omega_k^{r(l)} \quad (6)$$

Algorithm 1: L-DAWA: Let us Consider the Server Randomly Selecting K clients at the Given Round. The Clients Train the SSL Model With L Layers for E Local Epochs on Its Dataset d_k With n_k Number of Samples. The FL Optimization Lasts R Rounds.

```

1  Input:  $K, R, n_k, d_k, E, L$ 
2  Output:  $w_g^R$ 
3  Central server does:
4  for  $r = 1, \dots, R$  do
5  Server randomly selects  $K$  clients.
6  for  $k = 1, K$  do
7   $w_k^r, n_k, \mathcal{L}_k = \text{TrainLocally}(k, w_g^r, E)$ 
8  Aggregation:
9  for  $l = 1, \dots, L$  do
10 Compute divergence  $\delta_k^{(l)}$  based on Eq. (4).
11 If  $M$ -DAWA then: Compute  $w_g^{r+1}$  based on Eq. (5).
12 If  $L$  - DAWA then: Compute  $w_g^{r+1}$  based on Eq. (6).
13 If  $L$  - DAWAFedAvg or  $L$  - DAWAFedU then:
14 Compute  $w_g^{r+1}$  based on Eq. (7).
16 Compute  $w_g^{r+1}$  based on Eq. (8)
17 Train Locally ( $k, w_g^r$ ):
18   1:  $w_k^r, \mathcal{L}_k = \mathcal{F}_{SSL}(d_k, w_g^r, E)$ 
   2: Upload  $w_k^r, n_k, \mathcal{L}_k$  to the server.
```

Depending on the usage of FedAvg or Loss as the aggregation strategy, the above equation is represented as [42]:

$$\omega_g^{r+1(\text{FedAvg})} = \sum_{l=1}^L \sum_{k=1}^K \frac{n_k}{\sum_{k=1}^K n_k} \delta_k^{(l)} \omega_k^{r(l)} \quad (7)$$

$$\omega_g^{r+1(\text{Loss})} = \sum_{l=1}^L \sum_{k=1}^K \frac{\exp(-\mathcal{L}_k)}{\sum_{k=1}^K \exp(-\mathcal{L}_k)} \delta_k^{(l)} \omega_k^{r(l)} \quad (8)$$

L-Dawa extends DAWA by considering layer-wise divergence in neural networks, further refining the aggregation process. Li et al. proposed an extension to DAWA, improving aggregation by focusing on the impact of individual layers within deep learning models [125].

L-DAWA solves the issue of unregulated client model divergence in federated self-supervised learning (F-SSL) by using the weight of layer within the global model fusion process that reveals the divergence.

L-DAWA provides improved efficiency in the control of the FL process, especially within scenarios where data heterogeneity relates to a major problem. Restricting the method to layer-wise accumulation therefore guarantees that the overall model obtains the most appropriate layers from the local models in terms of both convergence rate and local optimum [41], [42]. This method is particularly useful in cross-silo and cross-device cases, where it identified that client models often have a high variance at the layer level. These enhancements have been supported in many tasks showing that L-DAWA significantly outperforms in contrast and non-contrast self-supervised learning models

like SimCLR and Barlow Twins, and datasets like CIFAR-10, CIFAR-100, and Tiny ImageNet. Additionally, L-DAWA has proven to reduce computational overhead compared to full model divergence calculations, making it a more efficient choice for large-scale FL environments.

C. MODEL-CENTRIC AGGREGATION

1) MODEL DISTILLATION

Distillation is the compression technique focused on replacing a complicated ML model with a simpler model that approximates the original. In ML, knowledge distillation (KL) or model distillation is the process of transferring knowledge from a large model to a smaller one. While large models (such as very deep neural networks or ensembles of many models) have more knowledge capacity than small models, this capacity might not be fully utilized [43]. The goal of model distillation is to transfer knowledge from the teacher to the student model in a way that retains high accuracy but with reduced computational complexity [44]. The process typically involves the student model learning the output predictions or intermediate representations of the teacher model, often using a softened version of the teacher's output probabilities (logits) as training targets.

Hinton et al. [127] introduced KD to compress ensemble models into a single lightweight model, demonstrating its efficacy on MNIST and in improving commercial acoustic systems. In FL, model distillation—often termed federated knowledge distillation (FKD)—is employed to reduce communication and computation overhead by transmitting logits or partial updates instead of full model weights [43]. Clients train local models and share soft predictions (e.g., class probabilities or embeddings), which the server aggregates to update a global student model [43]. This approach not only minimizes bandwidth usage but also supports personalization, enabling clients to deploy lightweight models aligned with device constraints. Despite these advantages, FKD may lead to precision loss due to incomplete representation of original model knowledge, especially under non-IID data conditions [44]. Additionally, it demands careful tuning of hyperparameters, such as temperature, to optimize transfer efficacy [43].

2) FEDERATED DISTILLATION

The primary idea behind adopting KD within FL framework was to enhance the quality of the information exchanged between the clients and the server [48]. Other researchers such as Li et al. [49], Sattler et al. [45], and Seo et al. [50] later explored KD under methods with FL known as FD. The approaches described are focused on the client-side model training, using the KD for shrinking of the FL base network and improving the results of the global model by knowledge transfer. Although they provide limited data privacy protection, challenges lie in the reduction of the information exchanged with the teacher model and addressing issues of

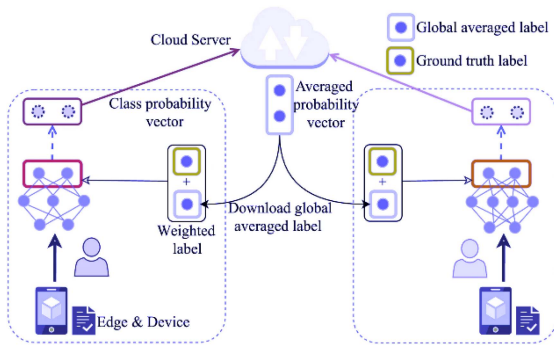


FIGURE 3. Federated distillation.

dataset heterogeneity. The rapid advancement of FD is evidenced by the growing volume of research papers published on the subject [51], [52], [53], [54].

FD is an example of performing model distillation in FL architecture as shown in Fig. 3. [45]. It allows clients to share new knowledge instead of sharing their models because the models are modified according to the local areas. In FD, every client updates its own model and transmits knowledge, for example output logits or soft labels, to the central coordinator [46]. All this knowledge is collected by the server and the information in the global model is then updated.

FD also ensures privacy because clients are not required to upload their model weights that may expose some information about their data [46]. In addition to this, FD is even more useful in situations where clients are different and thus have models of varied sizes or structure [47]. In this case, FD allows for knowledge exchange across the models without the need to harmonize the models structure among the clients. The existing FD methods concentrate on improving performance through the transfer of intermediate features or logits from the teacher model to the student model [55], [56], [57]. A variety of innovative modules and mechanisms have been proposed to enhance FD performance. These include synthetic sampling [58], bidirectional co-distillation [59], [60], [61], two-way mix-up [62], reinforcement distillation [63], [64], mutual distillation [65], Differential privacy (DP) [66], and meta-algorithm [67]. Additionally, numerous initiatives have been undertaken to implement FD across a range of applications, including wireless communications [68], keyword spotting [69], person re-identification [70], intelligent networked vehicle systems [71], intrusion detection [72], and classification [46].

In a classification task, FD mitigates the issue of common sample observations in Centralized Distillation (CD) by categorizing samples based on their labels, thus enhancing CD into a communication-efficient distributed learning framework [81] [82]. As depicted by Fig. 4, the operations of FD are summarized by the following four steps.

- Each worker stores a mean logit vector per label during local training.
- Each worker periodically uploads its local-average logit vectors to a parameter server averaging the uploaded

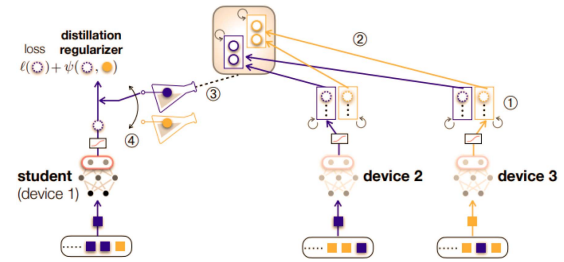


FIGURE 4. Illustration of FD with 3 devices and 2 labels in classification task.

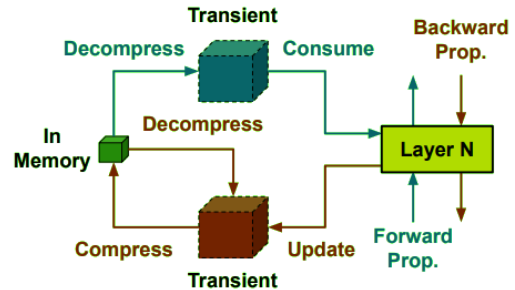


FIGURE 5. Model compression.

local-average logit vectors from all workers separately for each label.

- Each worker downloads the constructed global-average logit vectors of all labels from the server.
- During local training based on KD, each worker selects its teacher’s logit as the downloaded global-average logit associated with the same label as the current training sample’s ground-truth label.

The details of FD operations. Like CD, it is considered that the worker $c \in \{1, \dots, C\}$ has n observed samples with ground truth label, i.e., $\{\mathbf{x}_i^c, y_i^c\}_{i=1}^n$, but independently observed at each worker. For the sake of simplicity, assume $\mathcal{Y} = \{1, 2, \dots, |\mathcal{Y}|\}$ to be an alphabet of $|\mathcal{Y}|$ labels under consideration, and define an index set $\mathcal{I}_c^\downarrow$, that is composed of $\downarrow$ -labeled sample indices at the worker c , where $|\mathcal{I}_c^\downarrow| = n_c^\downarrow$ and $\sum_{\downarrow} n_{\downarrow}^c = n$. Under such circumstances, FD aims to solve the following optimization problem [50]:

$$\min_{\{\mathbf{w}_m^1\}, \dots, \{\mathbf{w}_m^c\}} \sum_c \sum_\ell \left(\underbrace{\sum_{i \in I_i^c} (y_i - \hat{y}_i^c)^2}_{\text{loss}} + \underbrace{\lambda \sum_{i \in I_\ell^c} \sum_m \left(\frac{1}{C-1} \sum_{c' \neq c} \tilde{f}_{m,l}^c - f_m^c(\mathbf{x}_i^c) \right)^2}_{\text{distillation regularizer}} \right) \quad (9)$$

where $f_m^c(\cdot)$ is the m -the logit of the worker c as before, $\bar{f}_{m,l} = \frac{1}{n_l^c} \sum_{i \in \mathcal{I}_l^c} f_m^c(\mathbf{x}_i^c)$ is the local average of the worker c 's

m -th logit for the samples labeled l . Following the four-step operations, FD solves the problem:

Algorithm: FD.

```

1 Require: Prediction:  $f(\mathbf{x})$ , Ground-truth label:  $y$ , Loss
  function:  $\mathcal{L}(f(\mathbf{x}), y)$ 
2 procedure LOCAL Training Phase (at worker  $\forall c \in$ 
   $\{1, \dots, C\}$ )
3 for  $k$  steps do :  $\mathcal{B} \leftarrow \mathcal{S}^c$ 
4 for sample  $\mathbf{x}_b$  and label  $y_b$ , for  $b \in \mathcal{B}$  do
5  $\mathbf{w}^c \leftarrow \mathbf{w}^c - \eta \nabla \{\mathcal{L}(f^c(\mathbf{x}_b), y_b) + \lambda \cdot \mathcal{L}(F^c(\mathbf{x}_b), \hat{F}_{y_b, r}^c)\}$ 
6  $F_{y_b, r}^c \leftarrow F_{y_b, r}^c + F^c(\mathbf{x}_b)$ ,  $\text{cnt}_{y_b, r}^c \leftarrow \text{cnt}_{y_b, r}^c + 1$ 
7 for label  $\hat{\downarrow} = 1, 2, \dots, |\mathcal{Y}|$  do
8  $\bar{F}_{\hat{\downarrow}, r}^c \leftarrow F_{\hat{\downarrow}, r}^c / \text{cnt}_{\hat{\downarrow}, r}^c$  return  $\bar{F}_{\hat{\downarrow}, r}^c$  to server
9 procedure GLOBAL ENSEMBLING PHASE (at
  the server
10 for each worker  $c = 1, 2, \dots, C$  do
11 for label  $\hat{\downarrow} = 1, 2, \dots, |\mathcal{Y}|$  do
12  $\bar{F}_{\hat{\downarrow}, r} \leftarrow \bar{F}_{\hat{\downarrow}, r} + \bar{F}_{\hat{\downarrow}, r}^c$ 
13   for each worker  $c = 1, 2, \dots, C$  do
14   for label  $\hat{\downarrow} = 1, 2, \dots, |\mathcal{Y}|$  do
15  $\hat{F}_{\hat{\downarrow}, r+1}^c \leftarrow \bar{F}_{\hat{\downarrow}, r} - \bar{F}_{\hat{\downarrow}, r}^c$ ,  $\hat{F}_{\hat{\downarrow}, r+1}^c \leftarrow$ 
 $\frac{\hat{F}_{\hat{\downarrow}, r+1}^c}{(C-1)}$  : return  $\hat{F}_{\hat{\downarrow}, r}^c$  to worker  $c$ 
16 end while

```

The set $\mathcal{S}^c$ denotes the training dataset of the worker c , and $\mathcal{B}$ represents a set of sample indices drawn as a batch per worker during the local training phase. The function $F^c(\cdot)$ is a logit vector, made by vectorizing the logits $\{f_m^c(\cdot)\}$. The function $\mathcal{L}(p, q)$ is a quadratic loss function, measuring the MSE between p and q , which is used for both loss function and distillation regularizer. Note that the quadratic loss can be replaced with any other well-defined loss function, such as cross-entropy [50]. As opposed to the asymptotic analysis, a constant learning rate η is assumed for practicality, and λ is a weighting constant for the distillation regularize. At the c -th worker, $\hat{F}_{\hat{\downarrow}, r}^c$ is the local-average logit vector at the r -th iteration when the training sample belongs to the $\hat{\downarrow}$ -th ground-truth label, $\bar{F}_{\hat{\downarrow}, r}^c$ is the global-average logit vector that equals $\hat{F}_{\hat{\downarrow}, r}^c = \sum_{c' \neq c} \bar{F}_{\hat{\downarrow}, r}^{c'} / (C - 1)$ with C workers, and $\text{cnt}_{\hat{\downarrow}, r}^c$ counts the number of samples whose ground-truth label is $\hat{\downarrow}$.

3) MODEL COMPRESSION

Model compression as shown in Fig. 5, is a critical area of research in ML aimed at optimizing models for deployment on resource-constrained devices. The increasing complexity of models and the constraints of real-world applications, including limited memory, computational power, and energy efficiency requirements, drive the need for model compression [84]. Effective model compression techniques can significantly reduce the size and complexity of neural networks while maintaining or even enhancing their performance. The main objectives of model compression include reducing storage requirements, increasing inference speed, lowering energy

consumption, and facilitating transfer learning. By minimizing the disk space needed to store models, enhancing the speed at which predictions can be made, and decreasing energy usage during model inference, practitioners can deploy efficient models on mobile applications and edge computing devices.

Due to the advancements in Big Data and GPU technology, deep learning, characterized by a centralized training paradigm, has been implemented across a range of applications including smart industrial systems, smart traffic management, and network threat detection. These applications are effective; however, it is essential that the neural network model undergoes prior training. This training process requires the collection of sample data from all clients or sensors via a wireless network. This process involves considerations related to privacy and transmission costs.

4) KNOWLEDGE TRANSFER

Knowledge transfer in ML is a key aspect that involves taking knowledge from one model and applying it to a different but comparatively smaller or less complex one [88]. In the field of FL, knowledge transfer improves the training of models by applying model trained at a larger data set to models trained at smaller local data set. The process often starts with training a source model which is also known as the teacher model on a large dataset. This model is capable of learning complex patterns, features and relationships of the data being fed to it. The knowledge of the teacher model is transferred to another model commonly known as the student model. This transfer can occur in several ways, with one of the most well-known being KD [89]. In this approach the teacher model output distribution is used during the training instead of the actual data distribution. The probabilities that the teacher model supplies, which can be softer probabilities over the classes, also provide extra information, which aids the student model to train in a better way. In this way, the student model learns to bound decisions like the teacher model and thus works better even on a limited dataset. The workflow of knowledge transfer can be outlined in a series of steps: First, a teacher model is trained to converge on a large diverse dataset of labels. Second, the outputs of the teacher model for the same inputs are gathered Third, the evaluations made by the students are acquired [88], [89]. Third, the student model is learned using these predictions; often the learning process involves utilizing both the original labels and the teacher's outputs. Lastly, to achieve good accuracy and generalization the performance of the student model is tested on a validation set. Using this workflow not only enables the small models training but also improves their performance and especially their generalization accuracy by Knowledge Transfer from the learned Teacher Model.

5) OTHER MODEL-CENTRIC METHODS

Besides knowledge transfer, many other model-centric approaches help to increase the efficiency and efficacy of the

model aggregation in FL scenario. There are many solutions and one of the most specific ones is efficiency-oriented aggregation that aims at reducing the amount of computational and communication flux required to put together and train the models from several devices. As aforementioned, in FL, local devices compute updates to their individual models from their own data [88]. These updates are then transmitted to an authoritative server where such updates are pieced together to form a global model. Some of the previous approaches like FedAvg implies averaging the model updates received from all the devices. Although this approach helps in consolidating the learned information, it entails a high communication overhead especially in large scales of devices or in such networks with a small bandwidth. Related to computation overhead, efficiency-oriented aggregation methods attempt to minimize this problem by providing efficient mechanisms of aggregating the values. Adaptive aggregation strategies involve following update frequency by weighing the information received from different devices; in this case, the update was weighted in terms of reliability and number density [89]. Thirdly, a variety of communication-efficient algorithms have been proposed to shrink the size of the updates which are sent, which can also make the aggregation process faster.

There is another group of sophisticated methods that are focused on security and stability during data aggregation. Additional Byzantine-tolerant aggregation schemes, for instance Krum and multi-Krum, aim at improving the quality of the aggregation by removing contaminated updates from the corrupt devices. In these approaches, the server gets the updates and then estimates the expected distribution of the metrics and removes metrics that are out of range [89]. This helps to make sure that the global model stays good regardless of attacks of adversarial instances and hardware imperfection. Hence, SMPC and HE would be incorporated with the aggregation process to ensure that information passing during learning is secure and protected.

D. ROBUST AND SECURE AGGREGATION

Secured aggregation is another important aspect of FL since as the number of devices increases so does the risk of encountering malicious behaviors. Frequently referred to as collaborative learning or distributed learning, FL can be described as a scenario in which multiple smart devices are used to train a centralized model while the data remain localized as shown in Fig. 6 [90]. But this structure opens the system to unreliable and jerky potentials from antagonistic individuals, which may distort the global model's reliability and efficiency. These risks are addressed by strong and secure aggregation methods which are used to prevent unfavorable influences from malicious or erroneous participants, but to ensure that the model updates in the global model are representative of the new accumulated knowledge [91].

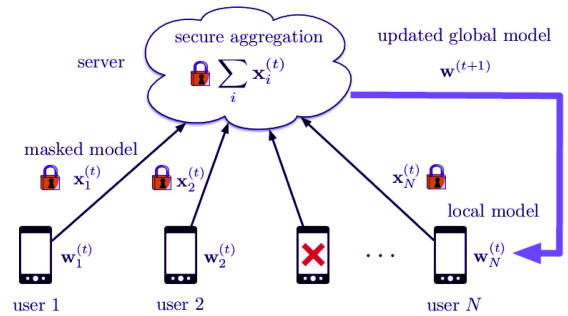


FIGURE 6. Secure aggregation in FL. At iteration t , the server sends the current state of the global model, denoted by $w(t)$, to the mobile users. User $i \in [N]$ forms a local model $w(t)_i$ by updating global model using its local dataset.

1) BYZANTINE-RESILIENT AGGREGATION

Byzantine-resilient aggregation has turned out to be one of the effective approaches towards the realization of the required FL robustness. This approach is particularly useful in Byzantine failures where some of the devices may give wrong or corrupted data updates either due to faulty or malicious behavior [92]. They prevent such detrimental updates from contaminating the aggregation computation while maintaining Byzantine-resilient aggregation methods as highly secure and stable in the presence of malicious stakeholders. In Byzantine-resilient aggregation, the maximally possible number of updates are provided by the trustworthy participants in the aggregation while allowing the exclusion of an extensive number of participants' updates, most of which can be faulty or malicious [93].

Traditional FL aggregation techniques are vulnerable to faulty or malicious clients, prompting the development of robust methods. KRUM, proposed by Blanchard et al. [37], is a Byzantine-resilient rule that selects client updates based on similarity. Its variant, MKRUM, improves convergence without sacrificing robustness. Median-based rules were introduced by Chen et al. [39] and Yin et al. [94] to mitigate outlier influence. Bulyan, by Mhamdi et al. [34], recursively applies robust rules like KRUM or COMED [94] for enhanced resilience against poisoning. A modular framework implementing such rules is described in [91], though communication inefficiencies and assumptions of equal data distribution remain concerns [32]. Zeno, introduced by Xie et al. [95], ranks client gradients and uses only the top- k for updates. Later variants remove reliance on a fixed k , enabling inclusion of all valid updates and improving performance when malicious clients are sparse. Despite these defenses, many robust methods remain vulnerable to adversarial attacks such as targeted poisoning [82], backdoors [83], and specific exploits against KRUM [27], COMED [23], and Bulyan [22]. Trimmed Mean [99] offers an alternative by discarding extreme updates, enhancing stability in settings with noisy or compromised devices, and producing smoother global model convergence, particularly useful in intrusion detection systems

(IDS). The algorithm for Trimmed Mean can be described as follows:

Algorithm: Trimmed Mean Aggregation (K.R. p).

```

1 Input: Number of devices, Rounds, Trim percentage
2 While K  $\neq$  length (ConnectedClients())do
3 loop
4 End while
5 TrimmedMeanAgg(p)
6 ReleaseClients ()
7 Procedures TrimmedMeanAgg(p)
8 Train on client k's dataset:  $\theta_t^k - \theta_t$ 
9 Compute local update:  $\delta\theta_t^k = \theta_t^k - \theta_t$ 
10 Send local update to server:  $\delta\theta_t^k$ 
11 end for
13 Receive local updates:
16 Sort updates:  $\delta\theta_t \leftarrow \text{Sort}(\delta\theta_t^k)$ 
17 Compute aggregated update:
     $\Delta\theta_t = \text{Mean}(\delta\theta_t[l+1 : |K| - l])$ 
18 Update global model parameters:  $\theta_{t+1} = \theta_t + \Delta\theta_t$ 

```

Byzantine-resilient aggregation methods are crucial in FL for defending against malicious clients that submit incorrect updates to compromise model integrity, particularly in sensitive applications like healthcare and smart grids. These techniques help preserve model performance and optimization efficiency by filtering out adversarial contributions in distributed environments.

Byzantine-resilient aggregation methods aim to mitigate adversarial behavior by filtering or adjusting malicious updates. However, this robustness often comes at a cost to model performance and computational overhead [92]. For instance, techniques such as Krum and Multi-Krum effectively defend against poisoning but may exclude benign but diverse updates, reducing generalization. Similarly, norm-clipping and median-based methods can dampen valuable gradients [93]. Trade-offs between security tolerance, statistical efficiency, and computational cost must therefore be considered, especially under diverse threat models such as omniscient or collusion-capable adversaries.

2) SECURE MULTIPARTY COMPUTATION (SMPC)

SMPC is a cryptographic technique whereby multiple IDS nodes or participants can perform a function while at the same time keeping their individual inputs private [100]. In FL, SMPC ensures that the updates or gradients as offered by the different IDS nodes are computed with security and privacy.

Every IDS node is equipped to make computations for model updates locally and do that while the raw data stays secure. Subsequently, the SMPC protocol integrates these updates in a secure manner to produce the final aggregated model update. This privacy-preserving method facilitates efficient collaboration among IDS nodes, even in situations where

participants may be untrusted or affiliated with different entities.

In FL, prevalent cryptography technologies encrypt the model parameter information that needs to be transmitted in plaintext. This process improves the performance of privacy and security protections within FL systems. Currently, the prevalent encryption techniques encompass SMPC and HE. Every encryption technology possesses distinct technical attributes. SMPC maintains the confidentiality of user input data while enabling multiple parties to conduct joint computations on confidential data; however, the associated computing overhead is significant. In comparison to SMPC, HE schemes exhibit comparable performance in terms of security protection. However, they utilize fewer computing resources compared to SMPC.

Kilbertus et al. in [101] employed SMPC methods for the purposes of validation and model training, ensuring that local confidential data remains confidential and is not accessible to other users. Sharemind, as presented by Bogdanov et al. in [102], developed a SMPC framework aimed at providing a secure and efficient computing environment. Kalapaaking et al. in [103] introduced a CNN-based FL framework that integrates secure multiparty computation (SMPC) for aggregation alongside cryptographic techniques for inference. Encrypted on-premises models were transmitted to the cloud for (SMPC)-based cryptographic aggregation, leading to the formation of an encrypted global model. The encrypted global model was subsequently returned to each edge server for localized training, which enhanced the model's accuracy. This solution addresses the FL privacy security issue within the IoT environment operating under 6th-generation networks, while maintaining both the accuracy of the model and the confidentiality of the model parameters. This solution did not account for the costs associated with multi-source heterogeneous data encryption processing and the consumption of communication resources within the IoT environment.

Li et al. in [104] developed a vertical FL-ring (VFL-R) intended for FL models that operate under constraints of limited communication resources and reduced computational capacity in the coordinators. VFL-R represents an innovative vertical FL framework that integrates a ring architecture to facilitate multi-party collaborative modeling. The VFL-R framework streamlined the complex communication architecture among all parties and offered safeguards against semi-honest attacks. Furthermore, it diminished the impact of coordinators within the modeling process. Berry and Komninos in [105] addressed the challenges associated with the sizable number of computational rounds required by the SMPC, as well as the elevated costs associated with data transmission between parties. An efficient optimization framework for a CNN utilizing SMPC has been proposed. This framework integrates multiple optimization techniques derived from the wider domain of privacy-preserving deep learning. The implementation incorporated batch normalization to enhance privacy preservation and utilized polynomial approximations for the activation functions.

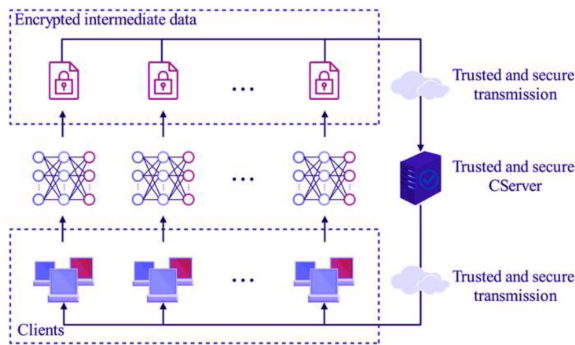


FIGURE 7. FL based on HE.

3) HOMOMORPHIC ENCRYPTION

This privacy technique allows the running of computations on encrypted content without having to decrypt the content first. In FL, this cryptographic technique helps in the safe addition of model updates while at the same time ensuring participant IDS holders did not disclose any of the raw data they possess. As noted in the training of collaborative model IDS nodes, every node encrypts the model update before initiating the transmission to the corresponding central server. Due to its ability to perform computations on the encrypted updates in this article, the server provides the final aggregated update without the client's raw data of the participants. This ensures that during the process data protection is enhanced in such a way that the privacy of individuals is protected while at the same time the IDS nodes may work interlinked [106].

In Fig. 7 HE-based schemes, users encrypt their models using a shared public key, enabling the server to aggregate updates via additive homomorphic properties. A key issue in such public-key systems is secret key management. As outlined in [106], three settings exist. In the first, users share a secret key unknown to the server, allowing all to access the global model. Common schemes include RSA [107], Paillier [108], lattice-based cryptosystems [109], BGN [110], and ElGamal [111]. In the second, the server holds the secret key, preserving the privacy of the global model [112], [113]. However, this setup compromises client privacy, as the server can decrypt user updates.

4) HE + FL (HE-FL)

Combining HE with FL allows computations to be performed directly on encrypted data without revealing raw information, making it highly suitable for privacy-critical applications such as healthcare and finance. Alghamdi et al. [106] proposes an optimized HE-based FL framework that minimizes encryption overhead while maintaining robust performance. The trade-off between security and computational cost remains a primary challenge in HE-FL systems.

5) ZERO-SHOT LEARNING IN FEDERATED AGGREGATION

Zero-shot learning (ZSL) enables model generalization across tasks with minimal or no target domain data, which is useful in

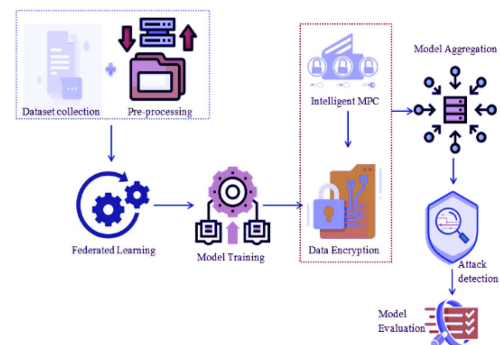


FIGURE 8. Hybrid FL framework.

FL scenarios with non-IID data or limited client participation. Integrate ZSL with FL to enhance model adaptability while preserving data privacy. This is particularly relevant for domains like cross-lingual NLP or disease prediction with rare class occurrences [129].

E. HYBRID AGGREGATION

The workflow of hybrid aggregation as shown in Fig. 8 is used to guide the aggregation process by also comprising data and model-based components. The usual first step of the procedure involves each client executing the local model training process using its data set [105]. Subsequently in the local training stage clients provide updates to a central server, but in contrast to data-centric or model-centric methods, the server performs a conjoint update step. In this stage, the server computes both the model parameters and ratio that is related to data distribution characteristics in each client. For instance, a client who has more, or different data may have its updates scaled higher than others, but the server also factors in, the efficiency of the update itself, CPU power or even adversarial behavior [107]. FedAvg, when used in conjunction with a model-centric approach such as model distillation, is an example of hybrid aggregation. In this setup, the server takes the averages of the Model updates using FedAvg and takes into consideration the numbers of data samples at the clients. Subsequently, the server subjects the accumulated model to distillation that makes the model even better by training it with those of the individual clients for better resilience and adaptability.

To enhance clarity, aggregation methods are categorized in Table 1, which summarizes their characteristics and categorization within the proposed taxonomy (data-centric, model-centric, robust, hybrid). This facilitates intuitive understanding of the design principles and comparative positioning of existing strategies.

IV. EVALUATION METRICS

As FL takes place in an open, decentralized, and heterogeneous environment, there is an evident need for comprehensive measurements which embrace a wide range of facets related to the model behavior and system performance and

TABLE 1. Categorization of Federated Aggregation Strategies

Method	Data-Centric	Model-Centric	Robustness-Oriented	Hybrid	Personalized	Key Attributes
FedAvg	✓	✓	✗	✗	✗	Simple averaging; baseline model
FedProx	✓	✓	✗	✗	✗	Regularizes local updates
Krum	✗	✗	✓	✗	✗	Byzantine resilience
Bulyan	✗	✗	✓	✗	✗	Aggregates using filtered mean
MOON	✗	✓	✗	✓	✓	Contrastive learning-based personalization
pFedMe	✓	✗	✗	✗	✓	Bi-level optimization for personalization
FedBABU	✓	✗	✗	✓	✓	Personalized adaptation via head training
RFA	✗	✓	✓	✓	✗	Uses geometric median for robustness

fairness [108]. Evaluation criteria comprise accuracy and convergence rate, communication overhead, both in terms of privacy and security, and both in terms of fairness and bias. These metrics answer how one or another aggregation strategy works in terms of potential performance and provide clues to select the most suitable methods for the defined tasks.

A. ACCURACY

Accuracy is among the essential forms of evaluation criteria across various milieux in FL. It shows the proficiency of the global model to calibrate results from unseen data on performance parameters. The objective of any aggregation strategy is to build a global model that performs well on other clients' datasets since first, the datasets cannot be IID (independent and identical distribution) and second, may not be balanced across the clients [102]. The quality is measured against common classification or regression metrics, and increased quality suggests the availability of a global model that fits better.

In classification tasks, accuracy is often measured using Top-1 accuracy, while additional metrics such as F1-score, precision, and recall are used to assess performance, particularly in imbalanced datasets. For regression tasks, accuracy is evaluated using mean squared error (MSE) or root mean squared error (RMSE). The effectiveness of FL aggregation directly impacts accuracy, and factors such as the quality of local model updates, the distribution of data across clients, and communication constraints play significant roles in determining the final model's performance. FL systems often encounter challenges due to Non-IID data, where clients have

highly heterogeneous datasets, leading to model drift and reduced accuracy if not properly addressed through aggregation techniques.

B. CONVERGENCE RATE

The convergence rate refers to the degree at which the global model achieves optimum performance or nearly optimum performance while training.

A slow convergence rate can significantly increase computational and communication overhead, making the system inefficient. The number of communication rounds required to achieve a target accuracy is often used to measure convergence, along with metrics such as loss reduction per iteration and the decay rate of gradient norms. Various factors impact the convergence rate, including the choice of learning rate, the level of heterogeneity in client data, and the number of participating clients in each aggregation round. Non-IID data distribution can slow down convergence because different clients contribute updates that do not align well with the global objective. FL aggregation methods such as FedAvg, FedOpt, and FedProx aim to mitigate slow convergence by adjusting the way local model updates are combined to improve learning stability across diverse client datasets.

Faster convergence is desirable in FL because it means the model attains the target quicker with fewer communication rounds, within a shorter amount of time and using less resources [112], [114]. The performance of aggregation strategies depends on the rate with which the model will reach a steady state. This is accrued from the fact that variants such as FedAvg are intentionally developed to converge by aggregating updates in a way that takes into consideration the local computation process and global synchronization process.

C. COMMUNICATION OVERHEAD

An essential factor in FL is communication overhead, as frequent client-server interactions occur. This cost, defined by data exchange between participants, impacts performance—especially with large models or many rounds [111]. Due to iterative updates, FL incurs more communication than centralized learning. Overhead is typically measured by bytes per client per round and total rounds to converge. Aggregation strategies directly affect this by shaping update frequency and format. Techniques like compression, quantization, and sparsification reduce update size with minimal performance loss. E.g., Federated Dropout prunes parameters before transmission, while FedAvg minimizes update frequency via local training. Asynchronous FL further enhances scalability by allowing clients to update at different times, reducing synchronization needs.

D. PRIVACY AND SECURITY

Privacy and security are fundamental concerns in FL, particularly when dealing with sensitive data such as healthcare records or financial transactions. While both concepts aim to protect data integrity, they address different aspects of protection. Privacy focuses on preventing data exposure, ensuring

that no party gains access to individual client information, whereas security aims to defend against adversarial threats that could compromise the learning process.

1) PRIVACY MECHANISMS IN FL

Privacy in FL is primarily maintained through techniques such as DP, HE, and SMPC. These mechanisms mitigate the risks of data exposure but introduce computational overhead, affecting FL's efficiency. DP ensures that individual client updates remain indistinguishable by adding controlled noise before aggregation, quantified using a privacy budget. While DP limits information leakage, it does not inherently secure the model against adversarial manipulations. HE allows computations to be performed on encrypted data, ensuring that raw data remains inaccessible. However, it does not provide a formal privacy guarantee and can be computationally expensive. SMPC enables multiple parties to jointly compute an aggregation function without revealing their individual data. While enhancing privacy, it demands additional computational resources. In addition to individual privacy-preserving techniques, collaborative frameworks have emerged to enable secure data collaboration among multiple parties. A notable approach is presented in [128], which proposes a privacy-preserving SMPC framework that complements secure aggregation techniques in FL. This framework integrates FL with cryptographic protocols and trust-aware coordination to achieve end-to-end privacy without exposing raw data or model parameters. Such solutions enhance collaborative intelligence while mitigating inference and collusion risks across distributed entities.

2) SECURITY THREATS AND COUNTERMEASURES

Security in FL focuses on defending against adversarial attacks, such as Byzantine attacks, where malicious clients inject poisoned updates to corrupt the global model. To counteract these threats, FL employs Byzantine-resilient aggregation techniques, including Krum and Multi-Krum, which filter out anomalous updates to maintain robustness. The Trimmed Mean method aggregates parameter updates while discarding extreme values, and robust FedAvg (RFA) improves aggregation resilience against malicious client updates. Unlike privacy-focused techniques, these methods mitigate adversarial influence rather than conceal client data. A robust FL system must integrate both privacy-preserving mechanisms and secure aggregation strategies to ensure data confidentiality and model integrity.

3) STRENGTHS OF PRIVACY-PRESERVING APPROACHES

The study by K. Wei et al. [126] provides an empirical evaluation of privacy-preserving FL and highlights key benefits. The successful integration of DP into FL demonstrates its applicability in real-world settings, particularly in privacy-sensitive domains such as healthcare. The use of (ϵ, δ) -DP ensures compliance with regulations like GDPR, offering a quantifiable

privacy guarantee [126]. DP helps mitigate risks from honest-but-curious adversaries who may attempt to infer information from parameter updates. Additionally, implementing DP via tools such as the Opacus module in PyTorch supports privacy-preserving ML practices in FL.

4) LIMITATIONS AND CHALLENGES

Despite its strengths, the study reveals several limitations in privacy-preserving FL approaches. First, applying DP often reduces model accuracy, particularly in non-IID and small dataset scenarios. For example, adding DP noise led to a 20% accuracy drop for MNIST, raising concerns about practical deployment in critical domains like medical diagnostics. Second, the effectiveness of DP depends on hyperparameter tuning [126]. A fixed clipping threshold and noise scale may not be optimal for different datasets, leading to instability in non-IID settings such as FEMNIST. Third, while DP protects individual records, it does not eliminate threats from model inversion and membership inference attacks, especially when the privacy budget (ϵ) is too high (e.g., $\epsilon = 100$). Finally, DP introduces overhead as clients must compute gradient perturbations before updating the global model. This challenge is particularly significant in cross-device FL, where computational capabilities vary widely.

5) POTENTIAL IMPROVEMENTS

To enhance privacy and security without compromising model performance, several strategies can be explored. Adaptive DP dynamically adjusts based on training progress and dataset characteristics to mitigate performance degradation. Hybrid privacy mechanisms, such as combining DP with HE or SMPC, could provide stronger guarantees against inference attacks while maintaining usability. Decentralized learning architectures, including d-clique topologies, could reduce reliance on a central server, minimizing privacy risks from aggregated updates. Adversarial training for Privacy can be incorporated to improve model resilience to inference attacks, supplementing DP without solely relying on noise-induced privacy preservation.

FL systems employ various privacy-preserving mechanisms including DP, SMPC, and HE. Each technique presents distinct trade-offs in terms of computational overhead and protection level. For instance, HE ensures high security but incurs higher latency. The following Table 2 provides a comparison of common privacy-preserving techniques used in FL:

These mechanisms highlight the dynamic interplay between privacy assurance and system efficiency.

E. FAIRNESS AND BIAS

Another essential characteristic for fairness is bias as clients in the FL process of different data distributions, computational capabilities, and local settings. This encompasses in the sense of fairness the extent to which the global model achieves

TABLE 2. Overview of Privacy-Preserving Techniques in FL

Technique	Strengths	Limitations	Use Cases
DP	Lightweight, scalable	Reduced accuracy at high noise	Mobile applications
SMPC	Involves multi-party	Communication overhead	Cross-institutional collaboration
HE	High privacy, no data decryption	Computationally expensive	Healthcare, Finance
HE + FL (HE-FL)	End-to-end encrypted computation	Latency and memory cost	Privacy-sensitive applications
ZSL in FL	Adaptability without labeled data	Performance on unseen domains	NLP, Image Recognition

acceptable performance on at least all clients and none of them have a higher or lower result than the others.

Fairness in FL is crucial due to varying client resources, data distributions, and participation rates. It is measured by evaluating model performance across clients to ensure accuracy doesn't disproportionately drop for underrepresented groups. Bias often stems from imbalanced data, where certain classes or demographics dominate some clients and are scarce in others. Aggregation methods must address this to prevent unfair outcomes. Weighted aggregation—based on data quality or frequency—can promote equity. Methods like FairFed and q-FedAvg introduce fairness constraints to optimize uniform performance rather than just global accuracy. Regularization and re-weighting also help ensure minority group representation in the global model. When key clients disproportionately influence global parameters, bias can degrade performance for smaller or underrepresented clients [117]. Hence, the effectiveness of aggregation strategies is analyzed based on their ability to fairly generalize a global model on all the client data including the outlying or minority data.

FL models may inadvertently propagate biases due to skewed local datasets or heterogeneous device capabilities. Strategies such as personalization layers, fairness-aware aggregation (e.g., q-FedAvg), and bias correction techniques are being explored to counteract these disparities. Future designs must integrate fairness constraints into optimization objectives to ensure equitable model performance across diverse user groups.

Table 3 presents a side-by-side comparison of secure aggregation methods based on key performance indicators to guide researchers and developers in selecting methods that align with their specific goals. For better understanding, Fig. 9 presents a decision-making framework for selection of averaging strategies.

F. STRUCTURED EVALUATION CRITERIA

To support consistent comparison of FL aggregation strategies, evaluation is structured across three core dimensions: model performance, privacy, and fairness.

TABLE 3. Comparative Summary of Secure Aggregation Methods

	Accuracy Retention	Privacy Guarantee	Bandwidth Efficiency	Computation Cost	Notes
SecAgg (Bonawitz et al.)	High	Information-Theoretic	Moderate	Moderate	Widely adopted; requires pairwise keys
HE-FL	Moderate	Strong (HE-based)	Low	High	High latency due to encryption cost
Secret Sharing + MPC	High	Strong (Honest Majority)	Low	High	Scales poorly with large clients
DP-SGD + Secure Aggregation	Moderate	DP (ϵ, δ)	High	Moderate	Balances utility and privacy
Lightweight Aggregation (e.g., SKA)	Slightly Reduced	Basic Encryption	High	Low	Practical in resource-constrained devices

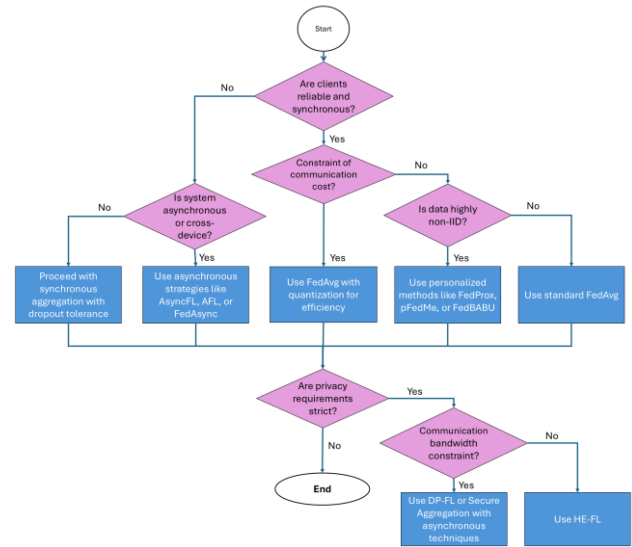


FIGURE 9. Decision-making framework of aggregation strategy.

Model performance is assessed using metrics such as accuracy, precision, recall, F1-score, and convergence speed on datasets like CIFAR-10, FEMNIST, and Shakespeare. In non-IID settings, additional indicators—such as variance in local model outcomes and global accuracy—help measure robustness.

Privacy is evaluated based on guarantees provided by DP, SMPC, and HE. Metrics include ϵ , δ , leakage rates, and added computation or communication overhead.

Fairness is analyzed by measuring client-wise performance variance, fairness-aware loss, and indices like Jain's index. Aggregation methods that address dominance by high-resource clients, such as Q-FedAvg and Agnostic FL, are prioritized for promoting equitable participation.

G. EVALUATION PROCEDURES FOR AGGREGATION METHODS

Rigorous comparison of FL aggregation strategies involves standardized procedures across five dimensions: accuracy, convergence, communication, fairness, and privacy. Accuracy and convergence are tracked under IID and non-IID conditions. Communication is measured per round and until convergence. Fairness is evaluated through client-level performance variation, and privacy through ϵ , δ values and resilience to adversarial interference. These procedures ensure reproducibility and clarify trade-offs between utility, efficiency, and security in diverse deployment settings.

V. FUTURE DIRECTIONS

As FL continues to evolve, several promising research directions are emerging, particularly around improving the security, efficiency, and personalization of aggregation strategies. These advancements aim to address the challenges of scaling FL systems, maintaining privacy, and ensuring that models are robust and fair in increasingly complex environments.

A. INNOVATIONS IN SECURE AND ROBUST AGGREGATION

With growing concerns over security, innovations in secure aggregation techniques are critical. Future developments in HE, SMPC, and DP aim to reduce the computational overhead while preserving the privacy of client data. Byzantine-resilient aggregation methods, such as advanced versions of Krum or Trimmed Mean, will focus on better handling malicious attacks by reducing the reliance on centralized verification and developing decentralized, fault-tolerant mechanisms. Further research is needed to combine these methods with efficient, lightweight cryptographic tools, making them scalable for large, dynamic networks.

Emerging techniques such as HE-FL and ZSL-FL reflect the direction toward making aggregation both secure and context-aware. HE-FL ensures end-to-end encryption without compromising model utility, while ZSL offers adaptability to unseen classes without client data sharing. These approaches will shape next-generation FL deployments.

B. IMPROVING EFFICIENCY IN LARGE NETWORKS

As FL networks grow, improving communication efficiency becomes paramount. Research will likely focus on gradient compression, quantization, and model pruning to reduce the size of updates transmitted between clients and servers. Innovations in asynchronous communication models, where clients communicate updates independently without waiting for synchronization across the entire network, can further boost efficiency in large-scale settings [119], [120]. Additionally, multi-tier hierarchical aggregation approaches, where intermediate aggregation is done at regional nodes before global aggregation, will gain traction to manage communication overhead and reduce latency.

Asynchronous aggregation has emerged as a promising solution to address the limitations of synchronous updates in large-scale federated networks. In contrast to synchronous methods that wait for all clients to upload their local updates, asynchronous strategies allow the server to proceed with partial updates. This improves communication efficiency and system throughput, especially under heterogeneous network conditions. Recent works have demonstrated that although asynchronous updates can introduce staleness in model parameters, various correction mechanisms (e.g., staleness-aware weighting, momentum compensation) can help retain convergence guarantees. These approaches contribute to enhanced scalability and robustness in cross-device FL environments.

C. HYBRID AGGREGATION METHODS

To balance competing demands such as accuracy, privacy, and efficiency, hybrid approaches that combine data-centric and model-centric aggregation methods are an area of active exploration. These approaches dynamically adapt the aggregation method based on real-time data characteristics, privacy requirements, and client resource availability. Hybrid models can combine elements from Adaptive Averaging, Federated Distillation, and Secure Aggregation techniques to adjust to distinct levels of heterogeneity in data or client participation. They specified it guarantees the homogeneity of results of the model across the world and at the same time allows providing local peculiarities by preserving privacy and high efficiency. Integration of data-centric and model-centric strategies, particularly through distillation and adaptive weighting, shows promise in accommodating client diversity while minimizing communication overhead.

D. EXPLORING PERSONALIZED AGGREGATION STRATEGIES

Some of FL requirements that have been identified include the following: when using FL, the data is always heterogeneous, and thus the importance of personalization [119]. Future work will further generalize such communication mechanisms so that clients can eschew using foreign models in lieu of global knowledge aggregation while still being able to retain exclusive local models optimized for their data. It is also expected that advanced methods, including federated averaging with personalization (FedAvg-Pers), and Clustered FL, will emerge to allow for a better fit of the global model for clients with unique data distributions or clusters of users with similar data distribution. Such techniques can solve problems of equality and prejudice; thus, the results obtained by a learning model will be equally accurate for various clients' datasets while being not limited by the perspective of overspecializing in the most significant sources of data. Personalized Federated Averaging (FedAvg-Pers), attention-based weighting, and meta-learning approaches are gaining traction for tailoring global models to user-specific distributions. Future work should prioritize scalable personalization mechanisms that can adapt to dynamic client behavior and task variation.

VI. CONCLUSION

Aggregation is a key component of FL and is used to contribute the local updated model that clients have to the global model. This survey has looked at numerous aggregation strategies including Simple Averaging, FedAvg, Adaptive Averaging, FD and Byzantine Resistant strategies including Krum and TM. Both strategies have their strengths and weaknesses, and each provides a different approach to balancing crucial factors such as accuracy, convergence rate communication overhead, privacy, and fairness. Even though easier methods like FedAvg have been popular due to performance and applicability, more complex methods such as Adaptive Averaging and Byzantine Resistant Aggregation deal with heterogeneity in the data and add robustness against adversarial data. The set of efficient aggregation strategies is the critical component to aim at for the real-world applicability of FL solutions, especially for tens or hundreds of thousands or millions of clients. As communication costs are usually the biggest issue in FL, techniques such as Model Compression, Quantization, and FD are crucial for reducing the size of the deltas and enhancing both scalability and convergence speed. Another reason for the demand for PIAP is that clients' data are frequently sensitive in various domains, including healthcare and finance. Such factors are resolved by privacy-preserving methods such as SMPC, and HE, though at a higher computational cost. The research studies identified existing research gaps in FL, including the need for more scalable and robust aggregation methods capable of reducing communication costs, enhancing privacy guarantees, and improving performance in highly heterogeneous environments. Besides, new research directions are highlighted, which will optimize secure aggregation to minimize the cost that comes with privacy-preserving technologies. The review synthesizes aggregation strategies by presenting both computational and privacy implications, supported by comparative frameworks and tabular analyses. Incorporating insights from recent contributions, this study equips practitioners with actionable guidelines for selecting appropriate aggregation methods based on security, performance, and scalability needs.

REFERENCES

- [1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Int. Conf. Artif. Intell. Statist.*, Apr. 2017, pp. 1273–1282. [Online]. Available: <http://proceedings.mlr.press/v54/mcmahan17a/mcmahan17a.pdf>
- [2] A. Hard et al., "FL for Mobile keyboard prediction," Jan. 2018, *arXiv:1811.03604*.
- [3] S. Ramaswamy, R. Mathews, K. Rao, and F. Beaufays, "FL for emoji prediction in a mobile keyboard," Jan. 2019, *arXiv:1906.04329*.
- [4] K. Bonawitz et al., "Towards federated learning at scale: System design," 2019, *arXiv:1902.01046*.
- [5] Q. Yang, Y. Liu, T. Chen, and Y. Tong, "Federated machine learning," *ACM Trans. Intell. Syst. Technol.*, vol. 10, no. 2, pp. 1–19, Jan. 2019, doi: [10.1145/3298981](https://doi.org/10.1145/3298981).
- [6] I. Ullah, N. U. Amin, A. Almogren, M. A. Khan, M. I. Uddin, and Q. Hua, "A lightweight and secured certificate-based proxy signcryption (CB-PS) scheme for E-prescription systems," *IEEE Access*, vol. 8, pp. 199197–199212, 2020.
- [7] J. P. Albrecht, "How the GDPR will change the world," *Eur. Data Protection Law Rev.*, vol. 2, no. 3, pp. 287–289, Jan. 2016, doi: [10.21552/edpl/2016/3/4](https://doi.org/10.21552/edpl/2016/3/4).
- [8] M. Parasol, "The impact of China's 2016 cyber security law on foreign technology firms, and on China's Big Data and Smart City dreams," *Comput. Law Secur. Rev.*, vol. 34, no. 1, pp. 67–98, Jun. 2017, doi: [10.1016/j.clsr.2017.05.022](https://doi.org/10.1016/j.clsr.2017.05.022).
- [9] R. Shokri and V. Shmatikov, "Privacy-preserving deep learning," in *Proc. 22nd ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2015, pp. 1310–1321, doi: [10.1145/2810103.2813687](https://doi.org/10.1145/2810103.2813687).
- [10] J. Konečný, H. B. McMahan, F. X. Yu, P. Richtárik, A. T. Suresh, and D. Bacon, "Federated learning: Strategies for improving communication efficiency," Jan. 2016, *arXiv:1610.05492*.
- [11] S. Caldas, J. Konečný, H. B. McMahan, and A. Talwalkar, "Expanding the reach of federated learning by reducing client resource requirements," Jan. 2018, *arXiv:1812.07210*.
- [12] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding," in *Proc. Int. Conf. Learn. Representations*, Feb. 2016. [Online]. Available: <https://forum.stanford.edu/events/posterslides/DeepCompressionCompressingDeepNeuralNetworkswithPruning,TrainedQuantizationandHuffmanCoding.pdf>
- [13] J. Xu, W. Du, Y. Jin, W. He, and R. Cheng, "Ternary compression for communication-efficient federated learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 3, pp. 1162–1176, Mar. 2022, doi: [10.1109/tnnls.2020.3041185](https://doi.org/10.1109/tnnls.2020.3041185).
- [14] Y. Chen, X. Sun, and Y. Jin, "Communication-efficient federated deep learning with layerwise asynchronous model update and temporally weighted aggregation," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 10, pp. 4229–4238, Oct. 2020, doi: [10.1109/tnnls.2019.2953131](https://doi.org/10.1109/tnnls.2019.2953131).
- [15] H. Zhu and Y. Jin, "Multi-objective evolutionary federated learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 4, pp. 1310–1322, Apr. 2020, doi: [10.1109/tnnls.2019.2919699](https://doi.org/10.1109/tnnls.2019.2919699).
- [16] Y. Zhao, M. Li, L. Lai, N. Suda, D. Cavin, and V. Chandra, "FL with Non-IID data," Jan. 2018, *arXiv:1806.00582*.
- [17] A. K. Sahu, T. Li, M. Sanjabi, M. Zaheer, A. Talwalkar, and V. Smith, "On the convergence of federated optimization in heterogeneous networks," Dec. 2018, *arXiv:1812.06127v1*.
- [18] S. Ji, S. Pan, G. Long, X. Li, J. Jiang, and Z. Huang, "Learning private neural language modeling with attentive aggregation," in *Proc. 2019 Int. Joint Conf. Neural Netw.*, Jul. 2019, pp. 1–8, doi: [10.1109/ijcnn.2019.8852464](https://doi.org/10.1109/ijcnn.2019.8852464).
- [19] L. T. Phong, Y. Aono, T. Hayashi, L. Wang, and S. Moriai, "Privacy-preserving deep learning via additively homomorphic encryption," *IEEE Trans. Inf. Forensics Secur.*, vol. 13, no. 5, pp. 1333–1345, May 2018, doi: [10.1109/tifs.2017.2787987](https://doi.org/10.1109/tifs.2017.2787987).
- [20] F. Armknecht et al., "A guide to fully homomorphic encryption," *IACR Cryptology ePrint Archive*, Sep. 2016. [Online]. Available: <https://ia.cr/2015/1192>
- [21] Y. Lindell and B. Pinkas, "Secure multiparty computation for privacy-preserving data mining," *J. Privacy Confidentiality*, vol. 1, no. 1, Apr. 2009, doi: [10.29012/jpc.v1i1.566](https://doi.org/10.29012/jpc.v1i1.566).
- [22] K. Bonawitz et al., "Practical secure aggregation for privacy-preserving machine learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2017, pp. 1175–1191, doi: [10.1145/3133956.3133982](https://doi.org/10.1145/3133956.3133982).
- [23] C. Dwork, "Differential Privacy: A survey of results," in *Proc. 5th Int. Conf. Theory Appl. Models Computation*, 2008, pp. 1–19, doi: [10.1007/978-3-540-79228-4_1](https://doi.org/10.1007/978-3-540-79228-4_1).
- [24] R. C. Geyer, T. Klein, and M. Nabi, "Differentially private FL: A client level perspective," Jan. 2017, *arXiv:1712.07557*.
- [25] T. Sun, D. Li, and B. Wang, "Decentralized federated averaging," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 4, pp. 4289–4301, Apr. 2023, doi: [10.1109/tpami.2022.3196503](https://doi.org/10.1109/tpami.2022.3196503).
- [26] K. Nishimoto, Y.-H. Chiang, H. Lin, and Y. Ji, "FedATM: Adaptive trimmed mean based federated learning against model poisoning attacks," in *Proc. IEEE 97th Veh. Technol. Conf.*, Jun. 2023, pp. 1–5, doi: [10.1109/vtc2023-spring57618.2023.10200010](https://doi.org/10.1109/vtc2023-spring57618.2023.10200010).
- [27] M. Tang and V. W. S. Wong, "An incentive mechanism for cross-silo federated learning: A public goods perspective," in *Proc. INFOCOM 2021 IEEE Conf. Comput. Commun.*, May 2021, pp. 1–10, doi: [10.1109/infocom42981.2021.9488705](https://doi.org/10.1109/infocom42981.2021.9488705).

- [28] Y. Wang, Y. Tong, D. Shi, and K. Xu, "An efficient approach for cross-silo federated learning to rank," in *Proc. IEEE 37th Int. Conf. Data Eng.*, Apr. 2021, pp. 1128–1139, doi: [10.1109/icde51399.2021.00102](https://doi.org/10.1109/icde51399.2021.00102).
- [29] J. So, B. Guler, and A. S. Avestimehr, "Byzantine-resilient secure federated learning," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 7, pp. 2168–2181, Jul. 2021, doi: [10.1109/jsac.2020.3041404](https://doi.org/10.1109/jsac.2020.3041404).
- [30] W. He, H. Yao, T. Mai, F. Wang, and M. Guizani, "Three-stage Stackelberg game enabled clustered federated learning in heterogeneous UAV swarms," *IEEE Trans. Veh. Technol.*, vol. 72, no. 7, pp. 9366–9380, Jul. 2023, doi: [10.1109/tvt.2023.3246636](https://doi.org/10.1109/tvt.2023.3246636).
- [31] Z. Qu, X. Li, J. Xu, B. Tang, Z. Lu, and Y. Liu, "On the convergence of multi-server federated learning with overlapping area," *IEEE Trans. Mobile Comput.*, vol. 22, no. 11, pp. 6647–6662, Nov. 2023, doi: [10.1109/tmc.2022.3200016](https://doi.org/10.1109/tmc.2022.3200016).
- [32] A. U. Khan, N. Javaid, M. A. Khan, and I. Ullah, "A blockchain scheme for authentication data sharing and nonrepudiation to secure internet of wireless sensor things," *Cluster Comput.*, vol. 26, no. 2, pp. 945–960, Apr. 2023.
- [33] J. Konečný, H. B. McMahan, D. Ramage, and P. Richtarik, "Federated optimization: Distributed machine learning for on-device intelligence," Jan. 2016, *arXiv:1610.02527*.
- [34] E. M. E. Mhamdi, R. Guerraoui, and S. L. A. Rouault, "The hidden vulnerability of distributed learning in Byzantium," in *Proc. Int. Conf. Mach. Learn.*, Jul. 2018, pp. 3521–3530.
- [35] A. N. Bhagoji, S. Chakraborty, P. Mittal, and S. B. Calo, "Analyzing federated learning through an adversarial lens," in *Proc. Int. Conf. Mach. Learn.*, May 2019, pp. 634–643. [Online]. Available: <http://proceedings.mlr.press/v97/bhagoji19a/bhagoji19a.pdf>
- [36] B. Biggio, B. Nelson, and P. Laskov, "Poisoning attacks against support vector machines," Jan. 2012, *arXiv:1206.6389*.
- [37] P. Blanchard, E. M. E. Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Proc. Neural Inf. Process. Syst.*, Dec. 2017, vol. 30, pp. 118–128. [Online]. Available: <https://papers.nips.cc/paper/6617-machine-learning-with-adversaries-byzantine-tolerant-gradient-descent.pdf>
- [38] L. Chen, H. Wang, Z. Charles, and D. Papailiopoulos, "DRACO: Byzantine-resilient distributed training via redundant gradients," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 903–912. [Online]. Available: <https://proceedings.mlr.press/v80/chen18l/chen18l.pdf>
- [39] Y. Chen, L. Su, and J. Xu, "Distributed statistical machine learning in adversarial settings," *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 1, no. 2, pp. 1–25, Dec. 2017, doi: [10.1145/3154503](https://doi.org/10.1145/3154503).
- [40] M. Wortsman et al., "Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 23965–23998. [Online]. Available: <https://proceedings.mlr.press/v162/wortsman22a/wortsman22a.pdf>
- [41] H. Park, L. L. Sjosund, Y. Yoo, N. Monet, J. Bang, and N. Kwak, "SINeT: Extreme lightweight portrait segmentation networks with spatial squeeze modules and information blocking decoder," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis.*, Mar. 2020, pp. 2055–2063, doi: [10.1109/wacv45572.2020.9093588](https://doi.org/10.1109/wacv45572.2020.9093588).
- [42] Y. A. Ur Rehman, Y. Gao, P. P. B. De Gusmão, M. Alibeigi, J. Shen, and N. D. Lane, "L-DAWA: Layer-wise divergence aware weight aggregation in federated Self-Supervised Visual Representation learning," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, Oct. 2023, pp. 16418–16427, doi: [10.1109/iccv51070.2023.01509](https://doi.org/10.1109/iccv51070.2023.01509).
- [43] Y. Guo, Z. Tan, and S. Lu, "Towards model-centric security for IoT systems," in *Proc. 29th Int. Conf. Comput. Commun. Netw.*, Aug. 2020, pp. 1–6, doi: [10.1109/icccn49398.2020.9209689](https://doi.org/10.1109/icccn49398.2020.9209689).
- [44] H. Amrani, "Model-centric and data-centric AI for personalization in human activity recognition," Doctoral Dissertation, Ph. D. Thesis, Univ. Milano-Bicocca, Milan, Italy. [Online]. Available: https://www.researchgate.net/profile/Hamza-Amrani-5/publication/355845434_Model-Centric_and_Data-Centric_AI_for_Personalization_in_Human_Activity_Recognition/links/61811f69a767a03c14e2e896/Model-Centric-and-Data-Centric-AI-for-Personalization-in-Human-Activity-Recognition.pdf
- [45] F. Sattler, A. Marban, R. Rischke, and W. Samek, "Communication-efficient federated distillation," Jan. 2020, *arXiv:2012.00632*.
- [46] T. Lin, L. Kong, S. U. Stich, and M. Jaggi, "Ensemble distillation for robust model fusion in federated learning," in *Proc. 34th Int. Conf. Neural Inf. Process. Syst.*, Jan. 2020, vol. 33, pp. 2351–2363.
- [47] H. Xing, Z. Xiao, R. Qu, Z. Zhu, and B. Zhao, "An efficient federated distillation learning system for multitask time series classification," *IEEE Trans. Instrum. Meas.*, vol. 71, 2022, Art. no. 2517012, doi: [10.1109/tim.2022.3201203](https://doi.org/10.1109/tim.2022.3201203).
- [48] E. Jeong, S. Oh, H. Kim, J. Park, M. Bennis, and S.-L. Kim, "Communication-efficient on-device machine learning: Federated distillation and augmentation under Non-IID private data," Jan. 2018, *arXiv:1811.11479*.
- [49] D. Li and J. Wang, "FEDMD: Heterogenous federated learning via model distillation," Jan. 2019, *arXiv:1910.03581*.
- [50] Y. C. Eldar, A. Goldsmith, D. Gündüz, and H. V. Poor, *Federated Knowledge Distillation*. Cambridge, U.K.: Cambridge Univ. Press, 2022, pp. 457–485, doi: [10.1017/9781108966559.019](https://doi.org/10.1017/9781108966559.019).
- [51] S. Itahara, T. Nishio, Y. Koda, M. Morikura, and K. Yamamoto, "Distillation-based semi-supervised federated learning for communication-efficient collaborative training with Non-IID private data," *IEEE Trans. Mobile Comput.*, vol. 22, no. 1, pp. 191–205, Jan. 2023, doi: [10.1109/tmc.2021.3070013](https://doi.org/10.1109/tmc.2021.3070013).
- [52] S. Han et al., "FedX: Unsupervised federated learning with cross knowledge distillation," in *Proc. Eur. Conf. Comput. Vis.*, in Lecture Notes in Computer Science, 2022, pp. 691–707, doi: [10.1007/978-3-031-20056-4_40](https://doi.org/10.1007/978-3-031-20056-4_40).
- [53] J. Kim, G. Kim, and B. Han, "Multi-level branched regularization for federated learning," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 11058–11073. [Online]. Available: <https://proceedings.mlr.press/v162/kim22a/kim22a.pdf>
- [54] H. Wen, Y. Wu, J. Li, and H. Duan, "Communication-efficient federated data augmentation on Non-IID data," in *Proc. 2022 IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops*, Jun. 2022, pp. 3376–3385, doi: [10.1109/cvprw56347.2022.00381](https://doi.org/10.1109/cvprw56347.2022.00381).
- [55] S. Cheng, J. Wu, Y. Xiao, and Y. Liu, "FEDGEMS: FL of larger server models via selective knowledge fusion," Jan. 2021, *arXiv:2110.11027*.
- [56] R. Mishra, H. P. Gupta, and T. Dutta, "A network resource aware federated learning approach using knowledge distillation," in *Proc. IEEE INFOCOM 2021 Conf. Comput. Commun. Workshops*, May 2021, pp. 1–2, doi: [10.1109/infocomwkshps51825.2021.9484597](https://doi.org/10.1109/infocomwkshps51825.2021.9484597).
- [57] J. Zhang, S. Guo, J. Guo, D. Zeng, J. Zhou, and A. Zomaya, "Parameterized knowledge transfer for personalized federated learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 10092–10104. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2021/file/5383c7318a3158b9bc261d0b6996f7c2-Paper.pdf
- [58] J. Zhang, S. Guo, J. Guo, D. Zeng, J. Zhou, and A. Zomaya, "Towards data-independent knowledge transfer in model-heterogeneous federated learning," *IEEE Trans. Comput.*, vol. 72, no. 10, pp. 2888–2901, Oct. 2023.
- [59] X. Ni, X. Shen, and H. Zhao, "Federated optimization via knowledge codistillation," *Expert Syst. Appl.*, vol. 191, Dec. 2021, Art. no. 116310, doi: [10.1016/j.eswa.2021.116310](https://doi.org/10.1016/j.eswa.2021.116310).
- [60] P. Qi, X. Zhou, Y. Ding, Z. Zhang, S. Zheng, and Z. Li, "FEDBKD: Heterogenous federated learning via bidirectional knowledge distillation for modulation classification in IOT-EDGE system," *IEEE J. Sel. Topics Signal Process.*, vol. 17, no. 1, pp. 189–204, Jan. 2023.
- [61] E. Shang, H. Liu, Z. Yang, J. Du, and Y. Ge, "FeDBIKD: Federated bidirectional knowledge distillation for distracted driving detection," *IEEE Internet Things J.*, vol. 10, no. 13, pp. 11643–11654, Jul. 2023, doi: [10.1109/jiot.2023.3243622](https://doi.org/10.1109/jiot.2023.3243622).
- [62] S. Oh, J. Park, E. Jeong, H. Kim, M. Bennis, and S.-L. Kim, "Mix2FLD: Downlink federated learning after uplink federated distillation with two-way MixUp," *IEEE Commun. Lett.*, vol. 24, no. 10, pp. 2211–2215, Oct. 2020, doi: [10.1109/lcomm.2020.3003693](https://doi.org/10.1109/lcomm.2020.3003693).
- [63] H. Cha, J. Park, H. Kim, S.-L. Kim, and M. Bennis, "Federated reinforcement distillation with proxy experience memory," Jan. 2019, *arXiv:1907.06536*.
- [64] H. Cha, J. Park, H. Kim, M. Bennis, and S.-L. Kim, "Proxy Experience Replay: Federated distillation for distributed reinforcement learning," *IEEE Intell. Syst.*, vol. 35, no. 4, pp. 94–101, Jul./Aug. 2020, doi: [10.1109/mis.2020.2994942](https://doi.org/10.1109/mis.2020.2994942).
- [65] Y. Li, W. Zhou, H. Mi, Y. Wang, and H. Wang, "FedH2L: A federated learning approach with model and statistical heterogeneity," in *Proc. IEEE Int. Conf. Joint Cloud Comput.*, Jul. 2023, pp. 25–33, doi: [10.1109/jcc59055.2023.00009](https://doi.org/10.1109/jcc59055.2023.00009).
- [66] Q. Li, B. He, and D. Song, "Practical one-shot federated learning for cross-silo setting," Jan. 2020, *arXiv:2010.01017*.

- [67] A. Taya, T. Nishio, M. Morikura, and K. Yamamoto, "Decentralized and model-free federated learning: Consensus-based distillation in function space," *IEEE Trans. Signal Inf. Process. Over Netw.*, vol. 8, pp. 799–814, 2022, doi: [10.1109/tsipn.2022.3205549](https://doi.org/10.1109/tsipn.2022.3205549).
- [68] H. Wen, Y. Wu, C. Yang, H. Duan, and S. Yu, "A unified federated learning framework for wireless communications: Towards privacy, efficiency, and security," in *Proc. IEEE INFOCOM 2020 Conf. Comput. Commun. Workshops*, Jul. 2020, pp. 653–658, doi: [10.1109/info-comwshps50562.2020.9162672](https://doi.org/10.1109/info-comwshps50562.2020.9162672).
- [69] A. Hard et al., "Production federated keyword spotting via distillation, filtering, and joint federated-centralized training," in *Proc. Interspeech 2022*, Sep. 2022, doi: [10.21437/interspeech.2022-11050](https://doi.org/10.21437/interspeech.2022-11050).
- [70] W. Zhuang et al., "Performance optimization of federated person re-identification via benchmark analysis," in *Proc. 28th ACM Int. Conf. Multimedia*, Oct. 2020, pp. 955–963, doi: [10.1145/3394171.3413814](https://doi.org/10.1145/3394171.3413814).
- [71] Z. Zhang, S. Wang, Y. Hong, L. Zhou, and Q. Hao, "Distributed dynamic map fusion via federated learning for intelligent networked vehicles," in *Proc. IEEE Int. Conf. Robot. Automat.*, May 2021, pp. 953–959, doi: [10.1109/icra48506.2021.9561612](https://doi.org/10.1109/icra48506.2021.9561612).
- [72] R. Zhao, Y. Wang, Z. Xue, T. Ohtsuki, B. Adebisi, and G. Gui, "Semisupervised federated-learning-based intrusion detection method for Internet of Things," *IEEE Internet Things J.*, vol. 10, no. 10, pp. 8645–8657, May 2023, doi: [10.1109/s11263-021-01453-z](https://doi.org/10.1109/s11263-021-01453-z).
- [73] C. Wang, G. Yang, G. Papanastasiou, H. Zhang, J. J. P. C. Rodrigues, and V. H. C. De Albuquerque, "Industrial cyber-physical systems-based cloud IoT EDGE for federated heterogeneous distillation," *IEEE Trans. Ind. Inform.*, vol. 17, no. 8, pp. 5511–5521, Aug. 2021, doi: [10.1109/tii.2020.3007407](https://doi.org/10.1109/tii.2020.3007407).
- [74] J. Gou, B. Yu, S. J. Maybank, and D. Tao, "Knowledge distillation: A survey," *Int. J. Comput. Vis.*, vol. 129, no. 6, pp. 1789–1819, Mar. 2021, doi: [10.1007/s11263-021-01453-z](https://doi.org/10.1007/s11263-021-01453-z).
- [75] Z. Li et al., "When object detection meets knowledge distillation: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 8, pp. 10555–10579, Aug. 2023, doi: [10.1109/tpami.2023.3257546](https://doi.org/10.1109/tpami.2023.3257546).
- [76] Y. Tian, S. Pei, X. Zhang, C. Zhang, and N. Chawla V., "Knowledge distillation on graphs: A survey," Jan. 2023, *arXiv:2302.00219*.
- [77] S. Samarakoon, M. Bennis, W. Saad, and M. Debbah, "Distributed federated learning for ultra-reliable low-latency vehicular communications," *IEEE Trans. Commun.*, vol. 68, no. 2, pp. 1146–1159, Feb. 2020, doi: [10.1109/tcomm.2019.2956472](https://doi.org/10.1109/tcomm.2019.2956472).
- [78] H. Kim, J. Park, M. Bennis, and S.-L. Kim, "Blockchain on-device federated learning," *IEEE Commun. Lett.*, vol. 24, no. 6, pp. 1279–1283, Jun. 2020, doi: [10.1109/lcomm.2019.2921755](https://doi.org/10.1109/lcomm.2019.2921755).
- [79] P. Kairouz et al., "Advances and open problems in federated learning," *Found. Trends Mach. Learn.*, vol. 14, pp. 1–120, 2021.
- [80] Z. Sun, H. Yu, X. Song, R. Liu, Y. Yang, and D. Zhou, "Mobile-BERT: A compact task-agnostic BERT for resource-limited devices," Jan. 2020, *arXiv:2004.02984*.
- [81] A. Jacot, F. Gabriel, and C. Hongler, "Neural tangent kernel: Convergence and generalization in neural networks (invited paper)," in *Proc. Adv. Neural Inf. Process. Syst.* 31, Jun. 2021, doi: [10.1145/3406325.3465355](https://doi.org/10.1145/3406325.3465355).
- [82] S. Zhou et al., "Distilling holistic knowledge with graph neural networks," in *Proc. 2021 IEEE/CVF Int. Conf. Comput. Vis.*, Oct. 2021, pp. 10367–10376, doi: [10.1109/iccv48922.2021.01022](https://doi.org/10.1109/iccv48922.2021.01022).
- [83] R. Anil, G. Pereyra, A. Passos, R. Ormandi, G. E. Dahl, and G. E. Hinton, "Large scale distributed neural network training through online distillation," Jan. 2018, *arXiv:1804.03235*.
- [84] C. Bucila, R. Caruana, and A. Niculescu-Mizil, "Model compression," in *Proc. 12th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, Aug. 2006, pp. 535–541, doi: [10.1145/1150402.1150464](https://doi.org/10.1145/1150402.1150464).
- [85] X. Zhu, L. Wang, W. Chen, and K. Sato, "Model compression and privacy preserving framework for federated learning," *Future Gener. Comput. Syst.*, vol. 140, pp. 376–389, Nov. 2022, doi: [10.1016/j.future.2022.10.026](https://doi.org/10.1016/j.future.2022.10.026).
- [86] J. Wu, L. Wang, and Y. Wang, "An improved CNN-LSTM model compression pruning algorithm," in *Proc. Adv. Natural Computation, Fuzzy Syst. Knowl. Discov.*, 2022, pp. 727–736, doi: [10.1007/978-3-030-89698-0_75](https://doi.org/10.1007/978-3-030-89698-0_75).
- [87] D. Alistarh, D. Grubic, J. Li, R. Tomioka, and M. Vojnovic, "QSGD: Communication-efficient SGD via gradient quantization and encoding," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 1707–1718.
- [88] Y. Pan, J. Ni, and Z. Su, "FL-PATE: Differentially private federated learning with knowledge transfer," in *Proc. 2021 IEEE Glob. Commun. Conf.*, Dec. 2021, pp. 1–6, doi: [10.1109/globe-com46510.2021.9685079](https://doi.org/10.1109/globe-com46510.2021.9685079).
- [89] L. Wang, C. Huang, and X. Han, "Vertical federated knowledge transfer via representation distillation," in *Proc. FL-IJCAI Workshop*, 2022. [Online]. Available: https://federated-learning.org/fl-ijcai-2022/Papers/FL-IJCAI-22_paper_18.pdf
- [90] P. Haghighi, P. Papadimitratos, M. Poturalski, K. Aberer, and J.-P. Hubaux, "Efficient and robust secure aggregation for sensor networks," in *Proc. 3rd IEEE Workshop Secure Netw. Protoc.*, Oct. 2007, pp. 1–6, doi: [10.1109/npsc.2007.4371623](https://doi.org/10.1109/npsc.2007.4371623).
- [91] G. Damaskinos, E.-M. El-Mhamdi, R. Guerraoui, A. Guirguis, and S. Rouault, "AGGREGATHOR: Byzantine machine learning via robust gradient aggregation," in *Proc. USENIX Workshop Tackling Comput. Syst. Problems Mach. Learn. Techn.*, Apr. 2019, vol. 1, pp. 81–106, doi: [10.5281/zenodo.2548779](https://doi.org/10.5281/zenodo.2548779).
- [92] Y. Xia, C. Hofmeister, M. Egger, and R. Bitar, "Byzantine-resilient secure aggregation for FL without privacy compromises," May 2024, *arXiv:2405.08698*.
- [93] E.-M. El-Mhamdi, R. Guerraoui, and S. Rouault, "Fast and robust distributed learning in high dimension," in *Proc. 2020 Int. Symp. Reliable Distrib. Syst.*, Sep. 2020, pp. 71–80, doi: [10.1109/srds51746.2020.00015](https://doi.org/10.1109/srds51746.2020.00015).
- [94] D. Yin, Y. Chen, K. Ramchandran, and P. L. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates," in *Proc. Int. Conf. Mach. Learn.*, Jul. 2018, pp. 5650–5659. [Online]. Available: <http://proceedings.mlr.press/v80/yin18a/yin18a.pdf>
- [95] C. Xie, O. Koyejo, and I. Gupta, "Zeno: Distributed stochastic gradient descent with suspicion-based fault-tolerance," in *Proc. Int. Conf. Mach. Learn.*, Jan. 2019, pp. 6893–6901.
- [96] L. Huang, A. D. Joseph, B. Nelson, B. I. P. Rubinstein, and J. D. Tygar, "Adversarial machine learning," in *Proc. 4th ACM Workshop Secur. Artif. Intell.*, Oct. 2011, pp. 43–58, doi: [10.1145/2046684.2046692](https://doi.org/10.1145/2046684.2046692).
- [97] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, "How to backdoor federated learning," in *Proc. Int. Conf. Artif. Intell. Statist.*, Jul. 2018, pp. 2938–2948. [Online]. Available: <http://proceedings.mlr.press/v108/bagdasaryan20a/bagdasaryan20a.pdf>
- [98] G. Baruch, M. Baruch, and Y. Goldberg, "A little is enough: Circumventing defenses for distributed learning," in *Proc. Neural Inf. Process. Syst.*, Jan. 2019, vol. 32, pp. 8632–8642. [Online]. Available: <https://papers.nips.cc/paper/9069-a-little-is-enough-circumventing-defenses-for-distributed-learning.pdf>
- [99] T. Wang, Z. Zheng, and F. Lin, "Federated learning Framework based on trimmed mean aggregation rules," *SSRN Electron. J.*, Jan. 2022, doi: [10.2139/ssrn.4181353](https://doi.org/10.2139/ssrn.4181353).
- [100] L. Zhou, L. Wang, Y. Sun, and T. Ai, "AntNest: Fully non-interactive secure multi-party computation," *IEEE Access*, vol. 6, pp. 75639–75649, 2018, doi: [10.1109/access.2018.2883775](https://doi.org/10.1109/access.2018.2883775).
- [101] N. Kilbertus, A. Gascón, M. J. Kusner, M. Veale, K. P. Gummadi, and A. Weller, "Blind Justice: Fairness with encrypted sensitive attributes," in *Proc. Int. Conf. Mach. Learn.*, Jul. 2018, pp. 2630–2639. [Online]. Available: <http://proceedings.mlr.press/v80/kilbertus18a/kilbertus18a.pdf>
- [102] D. Bogdanov, S. Laur, and J. Willemson, "ShareMind: A framework for fast privacy-preserving computations," in *Proc. 13th Eur. Symp. Res. Comput. Secur.*, 2008, pp. 192–206, doi: [10.1007/978-3-540-88313-5_13](https://doi.org/10.1007/978-3-540-88313-5_13).
- [103] A. P. Kalapaaking, V. Stephanie, I. Khalil, M. Atiquzzaman, X. Yi, and M. Almashor, "SMPC-based federated learning for 6G-enabled internet of medical things," *IEEE Netw.*, vol. 36, no. 4, pp. 182–189, Jul./Aug. 2022, doi: [10.1109/mnet.007.2100717](https://doi.org/10.1109/mnet.007.2100717).
- [104] J. Li, T. Yan, and P. Ren, "VFL-R: A novel framework for multi-party in vertical federated learning," *Appl. Intell.*, vol. 53, no. 10, pp. 12399–12415, Sep. 2022, doi: [10.1007/s10489-022-04111-0](https://doi.org/10.1007/s10489-022-04111-0).
- [105] C. Berry and N. Komninos, "Efficient optimisation framework for convolutional neural networks with secure multiparty computation," *Comput. Secur.*, vol. 117, Mar. 2022, Art. no. 102679, doi: [10.1016/j.cose.2022.102679](https://doi.org/10.1016/j.cose.2022.102679).
- [106] J. Park, N. Y. Yu, and H. Lim, "Privacy-preserving federated learning using homomorphic encryption with different encryption keys," in *Proc. 13th Int. Conf. Inf. Commun. Technol. Convergence*, Oct. 2022, doi: [10.1109/ictc55196.2022.9952531](https://doi.org/10.1109/ictc55196.2022.9952531).

- [107] Z. Liu, J. Guo, W. Yang, J. Fan, K.-Y. Lam, and J. Zhao, "Privacy-preserving aggregation in federated learning: A survey," *IEEE Trans. Big Data*, early access, Jul. 15, 2022, doi: [10.1109/tbdata.2022.3190835](https://doi.org/10.1109/tbdata.2022.3190835).
- [108] W. Yang, B. Liu, C. Lu, and N. Yu, "Privacy preserving on updated parameters in federated learning," in *Proc. ACM Turing Celebration Conf. - China*, May 2020, pp. 27–31, doi: [10.1145/3393527.3393533](https://doi.org/10.1145/3393527.3393533).
- [109] F. Tang, W. Wu, J. Liu, H. Wang, and M. Xian, "Privacy-preserving distributed deep learning via homomorphic re-encryption," *Electronics*, vol. 8, no. 4, Apr. 2019, Art. no. 411, doi: [10.3390/electronics8040411](https://doi.org/10.3390/electronics8040411).
- [110] D. Stripelis et al., "Secure neuroimaging analysis using federated learning with homomorphic encryption," in *Proc. 17th Int. Symp. Med. Inf. Process. Anal.*, Dec. 2021, doi: [10.1117/12.2606256](https://doi.org/10.1117/12.2606256).
- [111] D. Xu, S. Yuan, and X. Wu, "Achieving differential privacy in vertically partitioned multiparty learning," in *Proc. 2021 IEEE Int. Conf. Big Data*, Dec. 2021, pp. 5474–5483, doi: [10.1109/big-data52589.2021.9671502](https://doi.org/10.1109/big-data52589.2021.9671502).
- [112] C. Fang, Y. Guo, Y. Hu, B. Ma, L. Feng, and A. Yin, "Privacy-preserving and communication-efficient federated learning in Internet of Things," *Comput. Secur.*, vol. 103, Jan. 2021, Art. no. 102199, doi: [10.1016/j.cose.2021.102199](https://doi.org/10.1016/j.cose.2021.102199).
- [113] Y. Cheng, Y. Liu, T. Chen, and Q. Yang, "Federated learning for privacy-preserving AI," *Commun. ACM*, vol. 63, no. 12, pp. 33–36, Nov. 2020, doi: [10.1145/3387107](https://doi.org/10.1145/3387107).
- [114] K. Mandal and G. Gong, "PRIVFL: Practical privacy-preserving federated regressions on high-dimensional data over mobile networks," in *Proc. 2019 ACM SIGSAC Conf. Cloud Comput. Secur. Workshop*, 2019, pp. 57–68, doi: [10.1145/3338466.3358926](https://doi.org/10.1145/3338466.3358926).
- [115] D. Gao, Y. Liu, A. Huang, C. Ju, H. Yu, and Q. Yang, "Privacy-preserving heterogeneous federated transfer learning," in *Proc. IEEE Int. Conf. Big Data*, Dec. 2019, pp. 2552–2559, doi: [10.1109/big-data47090.2019.9005992](https://doi.org/10.1109/big-data47090.2019.9005992).
- [116] Y. Liu, Y. Kang, C. Xing, T. Chen, and Q. Yang, "A secure federated transfer learning framework," *IEEE Intell. Syst.*, vol. 35, no. 4, pp. 70–82, Jul./Aug. 2020, doi: [10.1109/mis.2020.2988525](https://doi.org/10.1109/mis.2020.2988525).
- [117] Y. Li, H. Li, G. Xu, X. Huang, and R. Lu, "Efficient privacy-preserving federated learning with unreliable users," *IEEE Internet Things J.*, vol. 9, no. 13, pp. 11590–11603, Jul. 2022, doi: [10.1109/jiot.2021.3130115](https://doi.org/10.1109/jiot.2021.3130115).
- [118] J. Ma, S. Naas, S. Sigg, and X. Lyu, "Privacy-preserving federated learning based on multi-key homomorphic encryption," *Int. J. Intell. Syst.*, vol. 37, no. 9, pp. 5880–5901, Jan. 2022, doi: [10.1002/int.22818](https://doi.org/10.1002/int.22818).
- [119] H. Zhu, R. Wang, Y. Jin, K. Liang, and J. Ning, "Distributed additive encryption and quantization for privacy preserving federated deep learning," *Neurocomputing*, vol. 463, pp. 309–327, Aug. 2021, doi: [10.1016/j.neucom.2021.08.062](https://doi.org/10.1016/j.neucom.2021.08.062).
- [120] K. Bonawitz et al., "Practical secure aggregation for federated learning on user-held data," Jan. 2016, *arXiv:1611.04482*.
- [121] S. J. Reddi et al., "Adaptive federated Optimization," May 2021, *arXiv:2003.00295*.
- [122] T. Nishio and R. Yonetani, "Client selection for federated learning with heterogeneous resources in mobile edge," in *Proc. 2019 IEEE Int. Conf. Commun.*, Shanghai, China, 2019, pp. 1–7.
- [123] Y. Chen, T. Zhang, X. Jiang, Q. Chen, C. Gao, and W. Huang, "Fedbone: Towards large-scale federated multi-task learning," 2023, *arXiv:2306.17465*.
- [124] H. Wang, M. Yurochkin, Y. Sun, D. Papailiopoulos, and Y. Khazaeni, "Federated learning with matched averaging," in *Proc. Int. Conf. Learn. Representations*, Apr. 2020. [Online]. Available: <https://www.openreview.net/pdf?id=BkluqlSFDS>
- [125] Q. Li, B. He, and D. Song, "Model-contrastive federated learning," in *Proc. 2022 IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2021, pp. 10708–10717, doi: [10.1109/cvpr46437.2021.01057](https://doi.org/10.1109/cvpr46437.2021.01057).
- [126] K. Wei et al., "Federated learning with differential privacy: Algorithms and performance analysis," *IEEE Trans. Inf. Forensics Secur.*, vol. 15, pp. 3454–3469, 2020, doi: [10.1109/tifs.2020.2988575](https://doi.org/10.1109/tifs.2020.2988575).
- [127] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," Mar. 2015, *arXiv:1503.02531*.
- [128] A. Salam, M. Abrar, F. Ullah, I. A. Khan, F. Amin, and G. S. Choi, "Efficient data collaboration using multi-party privacy preserving machine learning framework," *IEEE Access*, vol. 11, pp. 138151–138164, 2023, doi: [10.1109/ACCESS.2023.3339750](https://doi.org/10.1109/ACCESS.2023.3339750).
- [129] M. Asif et al., "Advanced zero-shot learning (AZSL) framework for secure model generalization in federated learning," *IEEE Access*, vol. 12, pp. 184393–184407, 2024, doi: [10.1109/ACCESS.2024.3510756](https://doi.org/10.1109/ACCESS.2024.3510756).